

AI-POWERED FAKE PRODUCT REVIEW DETECTION USING NLP

A PROJECT BASED LEARNING (PBL) REPORT

Submitted by

Santhosh (Register No. 2104251040010)

Aathithiyan (Register No. 2104251040020)

Submitted in partial fulfilment of the requirements

for the

Project-Based Learning component of the Machine Learning / NLP project

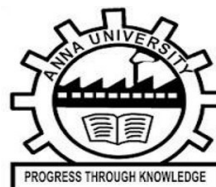
BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI



Affiliated to Anna University, Chennai
(Autonomous)

ACADEMIC YEAR 2026 – 2027

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

- IM1.** To create next generation leaders by effective teaching learning methodologies and in still Scientifics Park in them to meet the global challenges.
- IM2.** To transform lives through deployment of emerging technology, novelty and sustainability.
- IM3.** To inculcate human values and ethical principles to cater the societal needs.
- IM4.** To contribute towards the research ecosystem by providing a suitable,

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

- DM1.** To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.
- DM2.** To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.

BONAFIDE CERTIFICATE

This is to certify that the Project-Based Learning report titled “AI-Powered Fake Product Review Detection Using NLP” is a bonafide record of work carried out by Santhosh (Register No. 2104251040010) and Aathithiyan (Register No. 2104251040020) of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the mentor-guided Project-Based Learning (PBL) component of the Machine Learning / NLP project during the academic year 2026–2027 under my supervision.

SIGNATURE

.....
PROFESSOR AND HEAD,

Department of Computer Science and
Engineering
Chennai Institute of Technology,
Chennai – 69.

SIGNATURE

Mr. Pratham Verma, M.Tech.

FACULTY GUIDE

ASSISTANT PROFESSOR,
Department of Computer Science and
Engineering
Chennai Institute of Technology,
Chennai – 69.

Submitted for the final review held on

Internal Examiner

External Examiner

DECLARATION

We jointly declare that the Project-Based Learning report entitled “AI-Powered Fake Product Review Detection Using NLP” is the result of original work carried out by us under the guidance of Mr. Pratham Verma, M.Tech., Assistant Professor, Department of Computer Science and Engineering, Chennai Institute of Technology. To the best of our knowledge, this work has not been submitted, in part or in full, for the award of any other degree or diploma of Anna University, Chennai, or of any other university. Sources of data, software libraries and published literature used in this work are acknowledged in the report and listed in the References.

This report is submitted in partial fulfilment of the requirements of the Project-Based Learning component of the Bachelor of Engineering programme in Computer Science and Engineering.

Signature

Santhosh, Register No. 2104251040010

Aathithiyan, Register No. 2104251040020

Place: Chennai

Date:

ACKNOWLEDGEMENT

We express our sincere gratitude to the management of **Chennai Institute of Technology** for providing the academic environment, laboratory resources and computing facilities that made this Project-Based Learning work possible.

We thank the **Principal** and the **Dean** of the institution for their encouragement and for fostering a culture of project-oriented learning, in which students are expected to define a problem, build a working solution and evaluate it critically.

We are grateful to the **Head of the Department of Computer Science and Engineering** for the support and direction given to the PBL programme, and for the opportunity to pursue a project in Machine Learning and Natural Language Processing.

We extend our deepest thanks to our faculty guide, **Mr. Pratham Verma, M.Tech., Assistant Professor**, Department of Computer Science and Engineering, Chennai Institute of Technology, for the guidance given throughout the project. The guide's feedback on dataset validation, on the correct separation of training and test data, and on reporting results honestly shaped both the system and this report.

We also thank the faculty members and the technical and administrative staff of the department for their help during the review sessions, and our classmates for their comments on the application during testing. Finally, we acknowledge the authors and maintainers of the open-source tools used in this work — Python, Pandas, NumPy, NLTK, scikit-learn, Joblib and Streamlit — and the researchers who made the labelled fake-review dataset publicly available.

Santhosh (2104251040010)
Aathithiyan (2104251040020)

ABSTRACT

Online product reviews have become one of the main sources of information for consumers in e-commerce, and a product's ratings and written feedback strongly influence purchasing decisions. This influence also creates an incentive to publish fake reviews, including reviews produced automatically by computer programs. Such reviews are difficult to recognise by reading alone, and manual moderation or simple keyword rules do not scale to the volume of reviews posted every day.

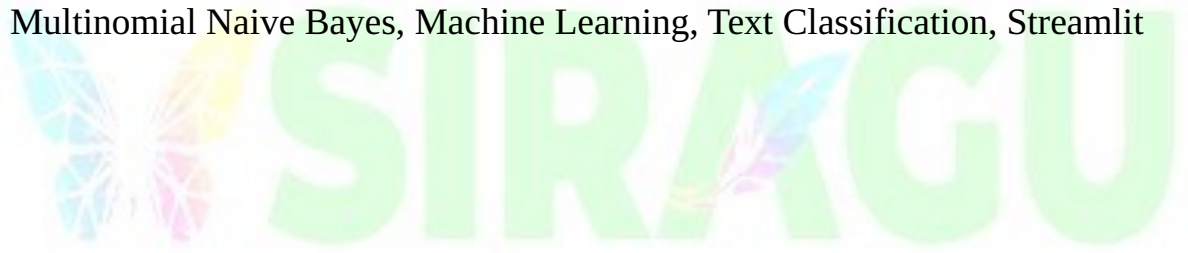
This project develops **AI-Powered Fake Product Review Detection Using NLP**, a lightweight and explainable machine-learning prototype that analyses the text of a customer review and classifies it as **Likely Fake** or **Likely Genuine**. The system is trained on a labelled dataset of 40,432 product reviews from ten product categories, evenly balanced between computer-generated (CG) and original (OR) reviews. Each review is preprocessed by lowercase conversion, removal of URLs, HTML tags, punctuation, digits and English stopwords, and whitespace normalisation. The cleaned text is converted into numerical features with **Term Frequency–Inverse Document Frequency (TF-IDF)** using unigrams and bigrams limited to 25,000 features, and a **Multinomial Naive Bayes** classifier with Laplace smoothing ($\alpha = 1.0$) is trained on these features. After removing duplicate review texts, the data were split by stratified sampling into 32,328 training and 8,083 test reviews.

On the unseen test set the model achieved an accuracy of **89.37%**, with a precision of 89.61%, a recall of 89.06% and an F1-score of 89.34% for the fake class; 3,598 fake and 3,626 genuine reviews were classified correctly, while 442 fake reviews were missed and 417 genuine reviews were flagged. The trained vectorizer and model are saved with Joblib and served through a

Streamlit web application that provides a landing page, a session-based login, a dashboard, review analysis with fake/genuine likelihoods and supporting language signals, side-by-side review comparison, session history, saved reviews and settings.

The prediction is a probabilistic, text-only indication and not proof of fraud. Its performance depends on the training data: the model learns the language of the computer-generated reviews in this dataset, works on English text, and does not examine reviewer identity, purchase history or product pages. Within these limits, the project demonstrates a complete, reproducible NLP pipeline from dataset validation to an interactive decision-support tool.

Keywords: Fake Review Detection, Natural Language Processing, TF-IDF, Multinomial Naive Bayes, Machine Learning, Text Classification, Streamlit



TEAM ROLES AND RESPONSIBILITIES

Division of roles and responsibilities between the two team members.

Team Member	Primary Role	Key Contributions
Santhosh (2104251040010)	Machine Learning, Backend and Model Development	Dataset loading and schema detection (<code>data_loader.py</code>), text preprocessing (<code>preprocessing.py</code>), the TF-IDF and Multinomial Naive Bayes training pipeline (<code>train_model.py</code>), evaluation metrics (<code>evaluate.py</code>), the inference module (<code>predict.py</code>) and saving of the model, vectorizer and metrics artifacts.
Aathithiyan (2104251040020)	Frontend, User Interface, Integration and Documentation	Streamlit application (<code>app.py</code>): landing, login/session, dashboard, review analysis, comparison, history, saved reviews and settings pages; presentation of predictions, likelihoods and language signals; integration of the interface with the inference module; project documentation.
Both members	Shared responsibilities	Dataset validation, testing of the model and the application, interpretation of the evaluation results, preparation of this report, the poster and the project presentation.

The division above describes where each member led the work; design decisions were discussed jointly, and both members share responsibility for the complete content of this report, including the methodology in Chapter 4, the implementation in Chapter 5 and the results in Chapter 6.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Driving Question.....	2
1.4 Objectives	3
1.5 Scope and Limitations	3
CHAPTER 2: CONCEPT EXPLORATION AND LITERATURE SURVEY	5
2.1 Fake Review Detection	5
2.2 Natural Language Processing.....	6
2.3 Text Classification.....	6
2.4 TF-IDF	7
2.5 Multinomial Naive Bayes	7
2.6 Related Approaches	7
2.7 Comparative Analysis.....	8
2.8 Summary	8
CHAPTER 3: PROJECT PLANNING AND TEAM ORGANISATION	10
3.1 Project Planning.....	10
3.2 Requirements	10
3.3 Functional Requirements.....	11
3.4 Non-Functional Requirements	12
3.5 Hardware and Software Requirements	12
3.6 Feasibility Study.....	12
3.7 Risk Management	13
3.8 Development Strategy	14
CHAPTER 4: DATASET AND MACHINE LEARNING METHODOLOGY .	15
4.1 Dataset Description.....	15
4.2 Dataset Validation.....	17
4.3 Data Preprocessing	18
4.4 Train/Test Split.....	20
4.5 TF-IDF Feature Extraction.....	20
4.6 Multinomial Naive Bayes	22

4.7 Model Training.....	23
4.8 Model Evaluation.....	24
4.9 Confusion Matrix.....	25
CHAPTER 5: SYSTEM DESIGN AND IMPLEMENTATION	27
5.1 System Architecture.....	27
5.2 Overall Workflow	28
5.3 NLP Pipeline.....	29
5.4 Module Description	30
5.5 Technology Stack	30
5.6 Model Implementation	31
5.7 Prediction Pipeline.....	32
5.8 Streamlit Application.....	33
5.9 User Interface.....	33
5.10 Project Folder Structure	40
CHAPTER 6: RESULTS AND DISCUSSION	41
6.1 Evaluation Methodology	41
6.2 Model Performance	41
6.3 Confusion Matrix.....	43
6.4 Classification Results.....	43
6.5 Application Testing	43
6.6 Discussion	45
6.7 Error Analysis	46
6.8 Limitations.....	47
6.9 Practical Interpretation.....	47
CHAPTER 7: TEAM REFLECTION AND LEARNING OUTCOMES	48
7.1 Individual Reflections.....	48
7.2 Team Learning	49
7.3 Technical Skills Acquired	49
7.4 Course Outcomes / Evidence Summary	50
CHAPTER 8: CONCLUSION AND FUTURE SCOPE.....	51
8.1 Conclusion	51
8.2 Future Scope.....	51

REFERENCES	53
APPENDIX	55
A.1 Project Folder Structure	55
A.2 Important Source Code	55
A.3 Model Training Configuration.....	56
A.4 Sample Prediction Workflow	56
POSTER.....	58



LIST OF FIGURES

Figure 4.1 Overall machine learning workflow of the project	15
Figure 4.2 Dataset class distribution (CG vs OR).....	16
Figure 4.3 Rating distribution of the dataset by label.....	17
Figure 4.4 Number of reviews per product category	17
Figure 4.5 NLP preprocessing pipeline implemented by clean_text().....	19
Figure 4.6 TF-IDF feature extraction workflow (fit on training data, transform for test and user text).....	21
Figure 4.7 Multinomial Naive Bayes classification workflow.....	23
Figure 4.8 Model training and evaluation pipeline (src/train_model.py).....	24
Figure 4.9 Confusion matrix on the 8,083-review test set (positive class = Fake).....	26
Figure 5.1 AI fake review detection system architecture	28
Figure 5.2 Overall application workflow.....	29
Figure 5.3 Model prediction pipeline (predict_review())	32
Figure 5.4 Streamlit landing page	34
Figure 5.5 How It Works page describing the four-step analysis.....	34
Figure 5.6 Login / session interface	35
Figure 5.7 Dashboard with session activity and recent analyses.....	36
Figure 5.8 Review analysis interface	36
Figure 5.9 Prediction result for the computer-generated-style sample (Likely Fake)...	37
Figure 5.10 Prediction result with language signals for the promotional-style sample.	38
Figure 5.11 Review comparison interface.....	38
Figure 5.12 Session analysis history with search and filter.....	39
Figure 5.13 Saved (bookmarked) reviews.....	39
Figure 5.14 Settings interface with model specification panel.....	40
Figure 5.15 Project folder structure.....	40
Figure 6.1 Model performance on the 8,083-review test set	42
Figure 6.2 Precision, recall and F1-score per class (from the saved classification report)	42

LIST OF TABLES

Table 2.1 Comparison of fake review detection approaches.....	8
Table 3.1 Project progress log.....	10
Table 3.2 Functional requirements.....	11
Table 3.3 Non-functional requirements	12
Table 3.4 Hardware and software requirements.....	12
Table 3.5 Risk management	13
Table 4.1 Dataset summary.....	15
Table 4.2 Label distribution.....	16
Table 4.3 Preprocessing operations	19
Table 4.4 Model configuration.....	23
Table 4.5 Evaluation metrics.....	25
Table 5.1 Module description.....	30
Table 5.2 Technology stack.....	30
Table 5.3 Project source files and responsibilities.....	31
Table 6.1 Model performance on the test set.....	41
Table 6.2 Confusion matrix values	43
Table 6.3 Application test cases	44
Table 6.4 Test results summary	45
Table 7.1 Technical skills and supporting evidence	49
Table 7.2 Learning outcomes and evidence	50

LIST OF ABBREVIATIONS

AI	– Artificial Intelligence
API	– Application Programming Interface
CG	– Computer-Generated (dataset label)
CSV	– Comma-Separated Values
F1	– F1-Score
FN	– False Negative
FP	– False Positive
IDF	– Inverse Document Frequency
ML	– Machine Learning
NB	– Naive Bayes
NLP	– Natural Language Processing
NLTK	– Natural Language Toolkit
OR	– Original (dataset label)
PBL	– Project-Based Learning
TF	– Term Frequency
TF-IDF	– Term Frequency–Inverse Document Frequency
TN	– True Negative
TP	– True Positive
UI	– User Interface
URL	– Uniform Resource Locator
UX	– User Experience

CHAPTER 1: INTRODUCTION

This chapter introduces the problem of fake product reviews, states the driving question and objectives of the project, and defines what the current implementation does and does not cover.

1.1 BACKGROUND

E-commerce has changed how products are discovered and bought. Instead of examining an item in a shop, a customer compares products on a screen, and the written reviews of earlier buyers have become one of the most important signals of quality. A review describes the experience of using a product — its fit, durability, packaging or after-sales service — in a way that a specification sheet cannot, and a large body of positive reviews makes a product more visible and more likely to be purchased. Research on online platforms has shown that review ratings have a measurable effect on sales and that this effect creates an incentive for sellers to manipulate reviews [4].

Fake reviews are reviews that do not reflect a genuine customer experience. They may be written by paid reviewers, by competitors, or produced automatically by computer programs. Early studies of opinion spam on review sites already showed that deceptive reviews are hard for human readers to detect [2], [3]. The arrival of language-generation models has made the problem harder, because a program can now produce many fluent, product-specific reviews at very low cost; a study on online product reviews found that such computer-generated reviews can be difficult for humans to tell apart from real ones [1].

Fake reviews damage the trust on which online shopping depends. A customer misled by inflated praise may buy an unsuitable product, honest sellers lose business to dishonest ones, and platforms lose credibility. Because the number

of reviews is very large and continuously growing, checking them by hand is not practical. This motivates the use of Natural Language Processing (NLP) and Machine Learning (ML) to examine the text of a review automatically and to indicate which reviews show the language patterns associated with fake content.

1.2 PROBLEM STATEMENT

A customer or a moderator who wants to know whether a review is trustworthy has few practical tools. Manual reading is slow, subjective and inconsistent between readers, and it cannot keep pace with the number of reviews published. Rule-based filters that search for words such as “best ever” or for excessive exclamation marks are easy to implement, but they are brittle: genuine enthusiastic customers trigger them, while fluent computer-generated reviews that avoid obvious phrases pass through. Fixed rules also do not learn from data and must be rewritten by hand whenever writing styles change.

The problem addressed by this project is therefore to build a system that *learns* the textual differences between computer-generated and original reviews from labelled examples, applies exactly the same processing to new reviews, reports its result as a likelihood rather than a certainty, and makes this analysis available through a simple web interface.

1.3 DRIVING QUESTION

How can a lightweight and explainable NLP-based machine-learning system automatically identify potentially fake product reviews from their textual characteristics while providing an accessible web interface for real-time analysis?

Answering this question required choosing a dataset with reliable labels, a preprocessing procedure that can be repeated identically at training and

prediction time, a feature representation and classifier that are fast and interpretable, an evaluation on data the model has never seen, and an interface that presents the result without overstating it.

1.4 OBJECTIVES

- Validate a labelled fake-review dataset (schema, missing values, duplicates, label balance) before training.
- Implement a consistent text-preprocessing function used both for training and for prediction.
- Represent review text with TF-IDF features using unigrams and bigrams.
- Train a Multinomial Naive Bayes classifier to label reviews as Fake (CG) or Genuine (OR).
- Evaluate the model on an unseen, stratified 20% test set using accuracy, precision, recall, F1-score and a confusion matrix.
- Save the trained model, the fitted vectorizer and the evaluation metrics so that results are reproducible.
- Provide a Streamlit web application for real-time review analysis that shows the prediction, class likelihoods and supporting language signals.
- Document the limitations of automated, text-only classification clearly.

1.5 SCOPE AND LIMITATIONS

Current implementation. The project implements: dataset validation and loading with automatic detection of the text and label columns; NLP preprocessing; a TF-IDF (1–2-gram, 25,000-feature) vectorizer and a Multinomial Naive Bayes model trained on the 40,432-review dataset; evaluation on 8,083 unseen test reviews; persistence of `model.pkl`, `vectorizer.pkl` and `metrics.json`; and a Streamlit application with landing, features, how-it-works and about pages, a session-based login, a dashboard, a review analyser, a two-review comparison, session history, saved reviews and

settings. All analysis is performed on the review text that the user types or pastes.

Limitations of the current implementation. The following are outside the present system and are stated so that they are not mistaken for delivered features:

- The classifier reads only the review text; it does not verify the reviewer, purchase records, timing or rating patterns.
- Product URLs or whole product pages are not fetched or analysed; reviews must be entered as text.
- The model was trained on English reviews from one dataset; behaviour on other languages or domains is not evaluated.
- The login is a session-only interface for demonstration: no passwords are stored or verified, and history and saved reviews exist only for the current browser session.
- No production deployment, load testing or user study was carried out.

Future scope. Product-page and URL analysis, multilingual detection, comparison with other classifiers and transformer models, explainability features and secure authentication are proposed as future work in Chapter 8; none of them is part of the current implementation.

CHAPTER 2: CONCEPT EXPLORATION AND LITERATURE SURVEY

This chapter reviews the concepts on which the project is built, surveys approaches to fake-review detection and explains why TF-IDF features with a Multinomial Naive Bayes classifier were selected.

2.1 FAKE REVIEW DETECTION

Fake-review detection, also studied as *opinion spam* detection, aims to identify reviews that do not describe a genuine experience. Jindal and Liu analysed large collections of product reviews and showed that duplicate and near-duplicate reviews and untruthful opinions are widespread [3]. Ott et al. built a corpus of deceptive hotel reviews and found that human judges performed close to chance, while classifiers using word n-gram features performed substantially better [2]. Salminen et al. studied reviews generated by language models for online products and released a dataset of computer-generated (CG) and original (OR) reviews across product categories [1]; the dataset used in this project follows the same CG/OR labelling scheme and category structure.

Three broad families of signals are used in the literature: **textual** signals from the review content, **behavioural** signals from the reviewer's activity (posting frequency, rating deviation, account age), and **relational** signals from links between reviewers, products and sellers. Behavioural and relational signals require platform data that is not available to an individual customer; this project therefore focuses on the textual signal, which can be computed from the review alone.

Manual review moderation

Platforms rely partly on human moderators and on customers reporting suspicious reviews. Human judgement can use context, but it is slow, costly

and inconsistent, and studies show that people are poor at recognising deceptive text unaided [2].

Keyword and rule-based detection

Rule-based filters flag reviews containing promotional phrases, excessive capitals or punctuation, or extreme length. Such rules are transparent and cheap, but they encode only what the rule author anticipated: they produce false alarms for genuine enthusiastic reviews and miss fluent generated reviews that contain no flagged phrase. The application in this project keeps a small set of such indicators, but only as supporting reading context — they do not decide the classification.

2.2 NATURAL LANGUAGE PROCESSING

Natural Language Processing is the field concerned with enabling computers to process and analyse human language [5]. For classical text classification, NLP provides the steps that turn free text into a consistent form: normalising case, removing markup and symbols, splitting text into tokens, and removing very frequent function words (stopwords) that carry little information about the class [6]. The Natural Language Toolkit (NLTK) supplies a standard English stopwords list, which is used in this project [13].

2.3 TEXT CLASSIFICATION

Text classification assigns a document to one of a set of predefined classes. In the supervised setting, a model is trained on documents whose classes are known and then applied to new documents [21]. A typical pipeline has three stages: preprocessing, feature extraction that represents each document as a numerical vector, and a classification algorithm that learns a decision rule from these vectors. Fake-review detection on this dataset is a binary text-classification problem with the classes Fake (CG) and Genuine (OR).

2.4 TF-IDF

TF-IDF (Term Frequency–Inverse Document Frequency) is a **feature extraction technique** that represents a document by weighted term counts [7], [9]. The term frequency measures how often a term occurs in a document; the inverse document frequency, introduced by Spärck Jones [8], reduces the weight of terms that appear in many documents and therefore distinguish little. Extending the vocabulary from single words (unigrams) to pairs of adjacent words (bigrams) lets the representation capture short phrases such as “highly recommend” or “works great”. TF-IDF is not itself a learning algorithm; it produces the input vectors that a classifier learns from.

2.5 MULTINOMIAL NAIVE BAYES

Multinomial Naive Bayes is a probabilistic **classification algorithm** that applies Bayes’ theorem under the assumption that features are conditionally independent given the class. The multinomial event model treats a document as a collection of term occurrences and was shown by McCallum and Nigam to be well suited to text classification [10]. Although the independence assumption is not literally true for language, Naive Bayes is known to perform well for text, trains in a single pass over the data, and has parameters (class priors and per-term likelihoods) that can be inspected directly. The scikit-learn implementation also accepts fractional TF-IDF values as input [11], [12].

2.6 RELATED APPROACHES

Logistic Regression and Support Vector Machines. Linear discriminative models trained on n-gram or TF-IDF features are strong baselines for text classification; Joachims showed that SVMs handle the many sparse features of text well [20]. They often match or exceed Naive Bayes in accuracy but require iterative optimisation and hyper-parameter tuning.

Transformer models. Pre-trained transformer models such as BERT [18] and RoBERTa [19] learn contextual representations of words and achieve high accuracy on many classification tasks, including the detection of machine-generated text. They require substantially more computation for training and inference, typically a GPU, and their decisions are harder to explain.

These approaches are discussed here for comparison only. **They were not trained or evaluated in this project;** comparing them with the present model is listed as future work.

2.7 COMPARATIVE ANALYSIS

Table 2.1 compares the approaches qualitatively. The entries describe typical characteristics reported in the literature, not measurements made in this project.

Table 2.1 Comparison of fake review detection approaches

Approach	Learns from data	Handles fluent generated text	Compute cost	Interpretability	Used in this project
Manual moderation	No	Poorly [2]	High (human effort)	High	No
Keyword / rule-based	No	Poorly	Very low	High	Supporting signals only
TF-IDF + Multinomial NB	Yes	Yes, if similar text is in training data	Low (CPU, seconds)	High (term probabilities)	Yes — final model
TF-IDF + Logistic Regression / SVM	Yes	Yes, if similar text is in training data	Low to moderate	Moderate (term weights)	No (future work)
Transformers (BERT, RoBERTa)	Yes	Generally strong	High (GPU recommended)	Low without extra tools	No (future work)

2.8 SUMMARY

The survey shows that fixed rules and manual reading are insufficient for fake-review detection, and that classifiers trained on word n-gram features are an

established and effective approach [2], [21]. For an academic PBL project with a balanced, labelled dataset of about forty thousand reviews, TF-IDF with Multinomial Naive Bayes was selected because it:

- is a well-understood baseline for text classification with a clear probabilistic interpretation;
- trains in seconds on a standard laptop CPU, which allows the complete pipeline to be re-run and verified during review sessions;
- produces class probabilities that can be shown to the user as likelihoods rather than as absolute verdicts;
- keeps the separation between the feature extraction technique (TF-IDF) and the classification algorithm (Naive Bayes) explicit, which suits the learning objectives of the course;
- creates small artifacts (under 1 MB each) that load quickly in a Streamlit application.

More complex models may achieve higher accuracy, but they would add computational cost and reduce explainability; they are therefore kept as future scope rather than presented as part of this work.

CHAPTER 3: PROJECT PLANNING AND TEAM ORGANISATION

This chapter records how the project was planned, the requirements of the system, its feasibility, the risks identified and the development strategy followed.

3.1 PROJECT PLANNING

The project was organised into weekly PBL milestones, each producing a result that could be checked before the next step began. Table 3.1 summarises the milestones; the Mentor Remarks column is reserved for the faculty guide.

Table 3.1 Project progress log

Week	Milestone / Task	Work Done	Mentor Remarks
Week 1	Problem identification and literature study	Defined the problem and driving question; studied opinion-spam research, TF-IDF and Naive Bayes; selected the labelled CG/OR dataset.	
Week 2	Dataset validation	Implemented <code>data_loader.py</code> ; verified 40,432 rows, 4 columns, 0 missing values, 12 duplicate rows, 20 duplicate texts and a 50:50 label balance.	
Week 3	Preprocessing and feature extraction	Implemented <code>clean_text()</code> ; configured the TF-IDF vectorizer (unigrams + bigrams, <code>min_df = 3</code> , 25,000 features) fitted on training data only.	
Week 4	Model training and evaluation	Trained Multinomial Naive Bayes ($\alpha = 1.0$); evaluated on the stratified 20% test set; saved model, vectorizer and metrics.	
Week 5	Application development	Built the Streamlit application: landing, login/session, dashboard, analyser, comparison, history, saved reviews and settings.	
Week 6	Testing, documentation and presentation	Tested input handling and navigation; analysed errors; prepared the report, poster and presentation.	

3.2 REQUIREMENTS

The requirements fall into five groups: *data* requirements (a labelled review dataset with a text column and a fake/genuine label), *ML* requirements (consistent preprocessing, feature extraction fitted only on training data, a

trained classifier and an evaluation on unseen data), *persistence* requirements (saving the model and vectorizer so predictions do not require retraining), *interface* requirements (a browser-based interface for entering reviews and reading results) and *reporting* requirements (presenting results as likelihoods with clear limitations).

3.3 FUNCTIONAL REQUIREMENTS

Table 3.2 Functional requirements

ID	Requirement	Implementation
FR1	Accept customer review text	Text area on the Analyze Review page; two text areas on the Compare page
FR2	Preprocess review text	<code>clean_text()</code> in <code>src/preprocessing.py</code>
FR3	Transform text using the saved TF-IDF vectorizer	<code>vectorizer.transform()</code> with <code>models/vectorizer.pkl</code>
FR4	Predict review class	<code>model.predict()</code> and <code>predict_proba()</code> with <code>models/model.pkl</code>
FR5	Display Likely Fake / Likely Genuine	Verdict banner on the analysis result
FR6	Display model-related analysis	Fake and Genuine likelihood percentages with progress bars; language signals
FR7	Provide a review analysis interface	Analyze Review page with sample reviews, word/character count, clear and analyse actions
FR8	Display model status / results where appropriate	“Model not trained” status when artifacts are missing; verified metrics on the landing page and in Settings

3.4 NON-FUNCTIONAL REQUIREMENTS

Table 3.3 Non-functional requirements

Attribute	How it is addressed
Usability	Navigation bar, sample reviews, word and character counts, plain-language verdicts
Maintainability	Separate modules for loading, preprocessing, training, evaluation and prediction; the UI calls a single function, <code>predict_review()</code>
Reliability	Empty input and missing artifacts return explicit status values instead of errors
Performance	Artifacts loaded once and cached in memory; prediction requires one sparse vector transform and one model call
Interpretability	Probabilistic model; likelihoods shown for both classes; language signals labelled as supporting context only
Data consistency	The same <code>clean_text()</code> is used in training and prediction; the vectorizer is fitted only on training data and reused unchanged

3.5 HARDWARE AND SOFTWARE REQUIREMENTS

Table 3.4 Hardware and software requirements

Category	Requirement
Processor / memory	A standard 64-bit laptop or desktop capable of running Python 3 and a modern web browser; no GPU is required (indicative requirement, not a measured benchmark)
Language	Python 3
Libraries	Pandas, NumPy, NLTK (English stopwords), scikit-learn, Joblib, Streamlit (versions as listed in <code>requirements.txt</code>)
Development tools	VS Code / Antigravity; Git
Client	Any current web browser (application served by Streamlit on <code>http://localhost:8501</code>)
Data	<code>dataset/fake_reviews_dataset.csv</code> (40,432 labelled reviews)

3.6 FEASIBILITY STUDY

Technical feasibility. All components are mature, open-source Python libraries with documented interfaces [11], [13], [14]. TF-IDF and Multinomial Naive Bayes are available in scikit-learn, and the dataset fits comfortably in memory.

Operational feasibility. The user only pastes a review and reads the result; no knowledge of machine learning is needed. Results are expressed as “Likely Fake” or “Likely Genuine” with likelihoods, which matches how a customer would use the information.

Economic feasibility. All software is free and open source, the dataset is publicly available, and training and inference run on an ordinary computer, so no hardware or licence costs are involved.

Schedule feasibility. The pipeline divides naturally into small, testable steps (validation, preprocessing, features, model, evaluation, interface) that fit the weekly PBL review cycle shown in Table 3.1.

3.7 RISK MANAGEMENT

Table 3.5 Risk management

Risk	Mitigation in the project
Dataset quality	<code>validate_dataset()</code> reports missing values, duplicate rows, duplicate texts, non-string values and label distribution before training
Class imbalance	Label distribution checked (exactly 50:50); stratified split preserves the ratio in training and test sets; a warning is raised for ratios above 4:1
Overfitting / data leakage	Duplicate review texts removed before splitting; TF-IDF fitted only on training data; evaluation only on the unseen 20% test set
Preprocessing mismatch	Single <code>clean_text()</code> function imported by both <code>train_model.py</code> and <code>predict.py</code>
Model loading failure	<code>is_model_trained()</code> checks for both artifacts; a “not trained” status is returned instead of an exception
Invalid user input	Empty input produces a warning and no prediction; text that becomes empty after cleaning falls back to the lower-cased raw text
Prediction inconsistency	Artifacts cached after first load; deterministic split (<code>random_state = 42</code>); metrics saved in <code>metrics.json</code>
UI / model integration issues	Interface depends only on the dictionary returned by <code>predict_review()</code> ; status values handled explicitly

3.8 DEVELOPMENT STRATEGY

Development was incremental and data-first. The dataset was validated before any model was built; preprocessing was written and inspected on sample reviews; the vectorizer and classifier were then trained and evaluated, and only after the metrics had been saved was the Streamlit application built on top of the saved artifacts. Keeping the ML code in the `src/` package and the interface in `app.py` allowed the model to be retrained without changing the interface, and the interface to be changed without retraining the model.



CHAPTER 4: DATASET AND MACHINE LEARNING METHODOLOGY

This chapter describes the dataset, its validation and preprocessing, and the machine-learning methodology: the train/test split, TF-IDF feature extraction, the Multinomial Naive Bayes classifier, training and evaluation. Figure 4.1 gives an overview.

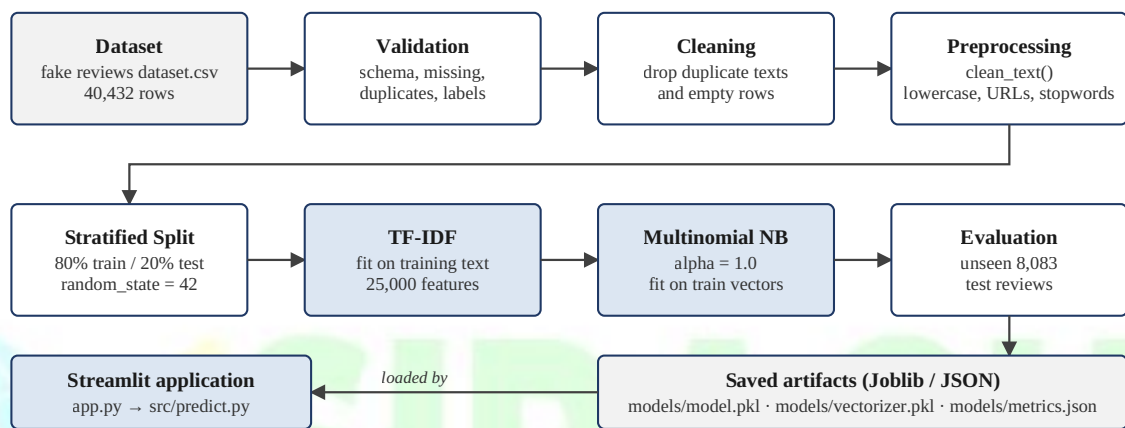


Figure 4.1 Overall machine learning workflow of the project.

4.1 DATASET DESCRIPTION

The project uses the file `dataset/fake reviews dataset.csv`, a labelled collection of product reviews in which each review is marked either **CG** (computer-generated, treated as fake) or **OR** (original, treated as genuine). The labelling scheme and the product categories correspond to the fake-review dataset described by Salminen et al. [1]. Table 4.1 summarises the verified properties of the file.

Table 4.1 Dataset summary

Property	Value
File	<code>fake reviews dataset.csv</code>
Total reviews (rows)	40,432
Total columns	4 — category, rating, label, text_
Review text column	text_

Property	Value
Label column	label (CG / OR)
Product categories	10
Rating range	1.0 – 5.0
Missing values	0
Exact duplicate rows	12
Duplicate review texts	20

Table 4.2 Label distribution

Label	Meaning	Mapped class	Reviews	Share
CG	Computer-Generated	Fake	20,216	50.0%
OR	Original	Genuine	20,216	50.0%
Total			40,432	100%

The dataset is exactly balanced (Figure 4.2). Ratings are skewed towards five stars, as is common for product reviews, but the two classes have very similar rating distributions (Figure 4.3); for example, 12,319 CG and 12,240 OR reviews carry a 5.0 rating. Rating alone therefore does not separate the classes, which supports the decision to classify on text. The reviews are spread across ten product categories with between 3,588 and 4,730 reviews each (Figure 4.4).

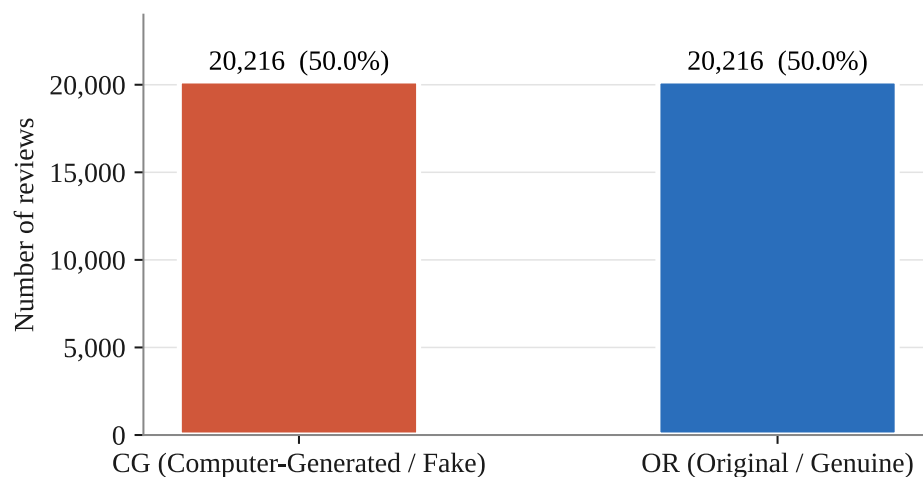


Figure 4.2 Dataset class distribution (CG vs OR).

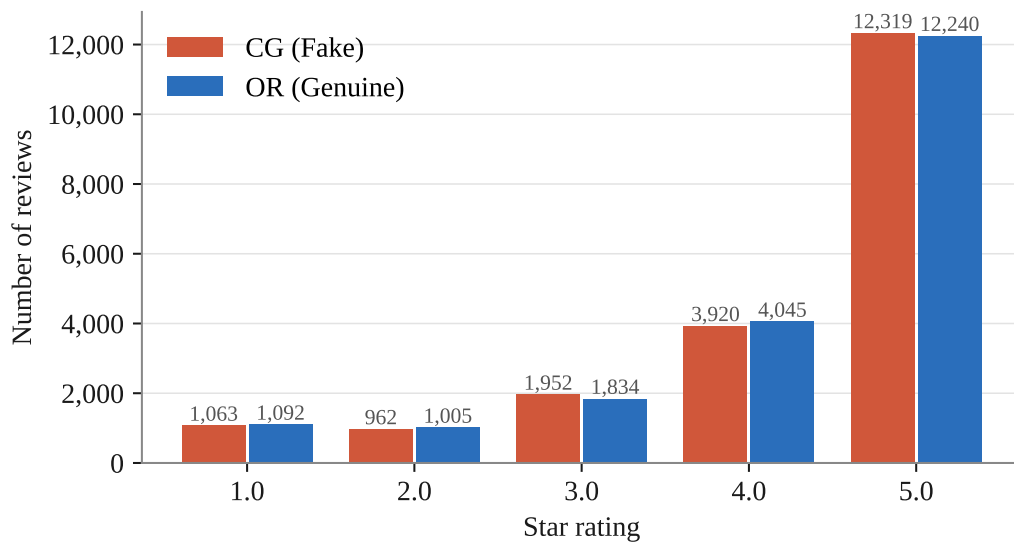


Figure 4.3 Rating distribution of the dataset by label.

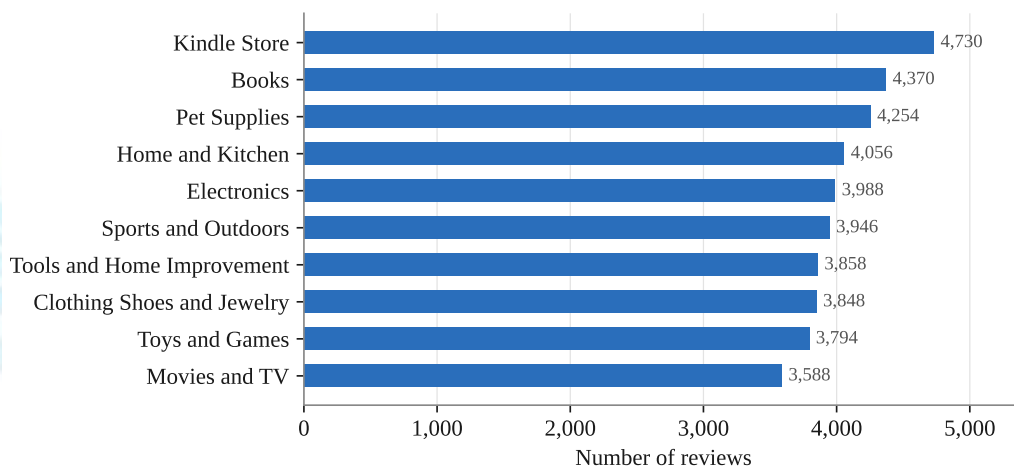


Figure 4.4 Number of reviews per product category.

4.2 DATASET VALIDATION

Before training, the dataset is checked by `validate_dataset()` in `src/data_loader.py`, which reads the file without modifying it. Because review datasets use different column names, the module does not hard-code them: it compares the column names, case-insensitively, against candidate lists for the text column (for example `review_text`, `review`, `text`, `text_`) and for the label column (for example `label`, `target`, `class`). For this file it detects `text_` and `label`, and reports category and rating as extra columns that are not used by the model.

The validation report records the number of rows and columns, missing values per column, exact duplicate rows, duplicate review texts, the unique labels and their counts, non-string values in the text column, and a class-imbalance warning when the largest class is more than four times the smallest. For this dataset it reported 0 missing values, 12 duplicate rows, 20 duplicate texts, the labels CG and OR with 20,216 reviews each, and no warnings.

When the dataset is loaded for training, `load_dataset()` keeps only rows with a non-empty text and a label and removes duplicate review texts, so that an identical review cannot appear in both the training and the test set. This leaves 40,412 reviews. After preprocessing, one further review became empty and was dropped, giving **40,411** usable reviews.

4.3 DATA PREPROCESSING

Raw review text contains variations in case, punctuation, symbols and very common words that add noise to the features. The function `clean_text()` normalises each review through the steps in Figure 4.5 and Table 4.3.

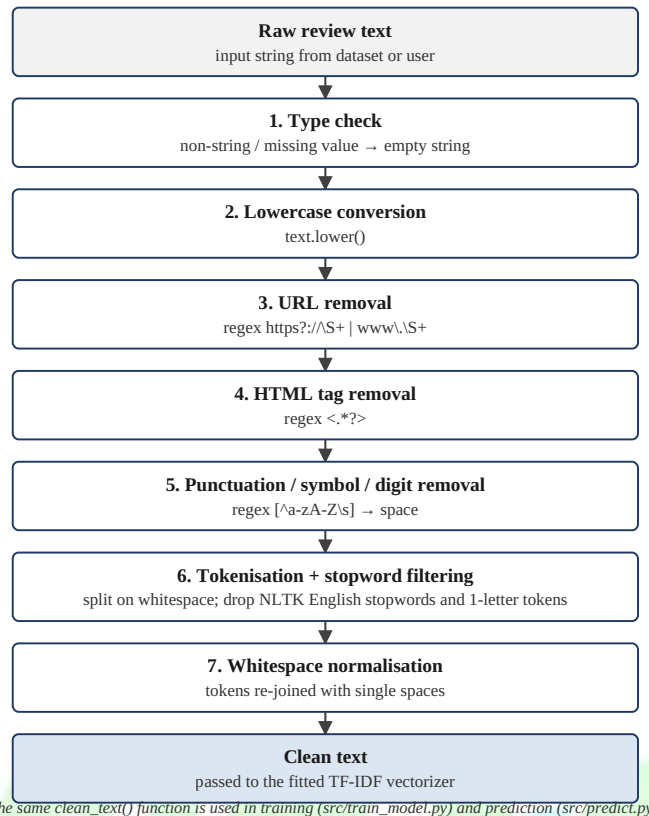


Figure 4.5 NLP preprocessing pipeline implemented by `clean_text()`.

Table 4.3 Preprocessing operations

Step	Operation	Purpose
1	Return an empty string for missing / non-string values	Handle missing values safely
2	Convert to lowercase	Treat “Great” and “great” as the same term
3	Remove URLs (<code>https?://\S+ www\.\S+</code>)	Links carry no review language
4	Remove HTML tags (<code><.*?></code>)	Remove markup left from web pages
5	Replace every non-alphabetic character with a space	Remove punctuation, symbols and digits
6	Split on whitespace; drop NLTK English stopwords and one-letter tokens	Keep content-bearing words
7	Join remaining tokens with single spaces	Normalise whitespace

For example, applying `clean_text()` to the illustrative input “*This product is AMAZING!!! Best purchase ever, visit www.deals-shop.com
 10/10 would buy again :)*” returns *product amazing best purchase ever visit would buy*: the URL, tag, digits and punctuation are removed, and the stopwords *this, is*

and *again* are dropped. The same function is imported by the training pipeline and by the prediction module, so a review entered in the application is processed exactly as the training reviews were. If a user review becomes empty after cleaning (for example, a review consisting only of punctuation), the prediction module falls back to the lower-cased raw text.

4.4 TRAIN/TEST SPLIT

The 40,411 usable reviews are divided with scikit-learn's `train_test_split` using `test_size = 0.2`, stratify on the label and `random_state = 42`. Stratification keeps the 50:50 class balance in both parts, and the fixed random state makes the split reproducible. The result is **32,328 training reviews** (16,156 Fake, 16,172 Genuine) and **8,083 test reviews** (4,040 Fake, 4,043 Genuine). The test reviews are not used in any way while fitting the vectorizer or the model.

4.5 TF-IDF FEATURE EXTRACTION

Machine-learning classifiers operate on numbers, so each cleaned review is converted into a numerical vector using TF-IDF, the **feature extraction technique** of this project (Figure 4.6).

Term Frequency (TF). For a term t and a document d , $tf(t, d)$ is the number of times t occurs in d . A term used repeatedly in a review is more representative of that review.

Inverse Document Frequency (IDF). Terms that occur in almost every review, such as *product*, help little to tell reviews apart. With the scikit-learn default (smoothed) formulation used here, $idf(t) = \ln[(1 + n) / (1 + df(t))] + 1$, where n is the number of training documents and $df(t)$ is the number of training documents containing t [12].

TF-IDF weighting. The weight of a term in a review is $tf(t, d) \times idf(t)$; each review vector is then normalised to unit length (L2 norm) so that long and short reviews are comparable [7].

Configuration. The vectorizer uses `ngram_range = (1, 2)`, so both unigrams and bigrams are features; `min_df = 3`, so a term must occur in at least three training reviews; and `max_features = 25,000`, keeping the 25,000 most frequent terms. The fitted vocabulary contains exactly 25,000 features.

Fitting only on training data. The vectorizer is fitted with `fit_transform()` on the 32,328 training reviews only. The test reviews, and every review entered in the application, are converted with `transform()` using the same fitted vocabulary and IDF weights. Terms never seen in training are ignored. Fitting on the test data would let information about the test set leak into the features and would make the evaluation optimistic.

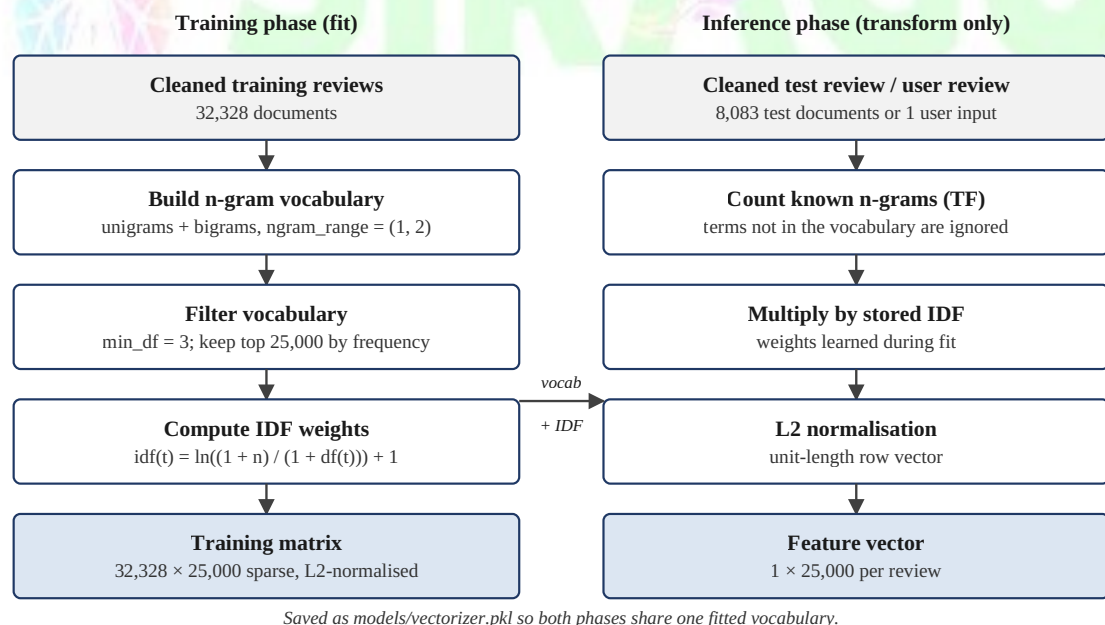


Figure 4.6 TF-IDF feature extraction workflow (fit on training data, transform for test and user text).

4.6 MULTINOMIAL NAIVE BAYES

Multinomial Naive Bayes is the **machine-learning classification algorithm** of this project. It is based on Bayes' theorem:

$$P(c | d) = P(c) \cdot P(d | c) / P(d)$$

where c is a class (Fake or Genuine) and d is a review. **Class probabilities.** The prior $P(c)$ is estimated from the proportion of training reviews in each class; with a balanced training set both priors are close to 0.5. **Word-feature probabilities.** Assuming the features are conditionally independent given the class, $P(d | c)$ is the product of the per-term likelihoods $P(w_i | c)$, each raised to the feature value x_i . The likelihoods are estimated from the summed TF-IDF weights of each term in each class [10]:

$$P(w_i | c) = (N_{c,i} + \alpha) / (N_c + \alpha \cdot |V|)$$

where $N_{c,i}$ is the total weight of term i in class c , N_c is the total weight of all terms in class c , and $|V| = 25,000$ is the vocabulary size. **Laplace smoothing.** The constant α adds a small pseudo-count to every term so that a term absent from one class does not produce a zero probability that would cancel all other evidence. The project uses $\alpha = 1.0$ (Laplace smoothing).

Classification of text documents. To classify a review, the model computes, for each class, $\log P(c) + \sum x_i \log P(w_i | c)$ and selects the class with the larger score (Figure 4.7). Normalising the two scores gives the class probabilities returned by `predict_proba()`, which the application shows as the Fake and Genuine likelihoods.

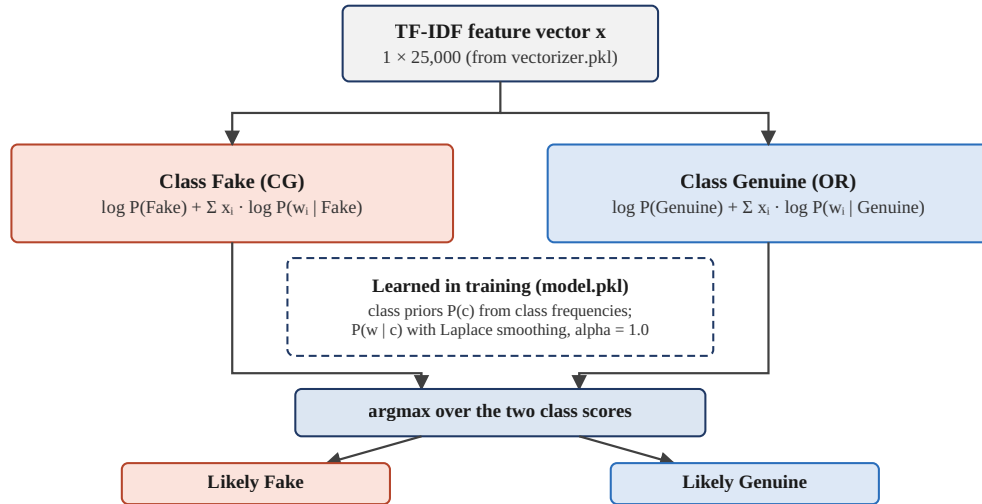


Figure 4.7 Multinomial Naive Bayes classification workflow.

Table 4.4 Model configuration

Component	Setting
Feature extraction technique	TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_features=25000); other parameters at scikit-learn defaults (L2 norm, smoothed IDF)
Classification algorithm	MultinomialNB(alpha=1.0) — Laplace smoothing
Label mapping	CG → Fake, OR → Genuine
Split	80% / 20%, stratified, random_state = 42
Samples	40,411 usable reviews → 32,328 training, 8,083 test
Vocabulary size	25,000 unigram and bigram features
Artifacts	models/model.pkl, models/vectorizer.pkl, models/metrics.json

4.7 MODEL TRAINING

Training is performed by `train()` in `src/train_model.py`, run with `python -m src.train_model`. The function prints each of its seven stages and then saves the artifacts (Figure 4.8): the dataset is loaded and de-duplicated, labels are mapped, every review is cleaned, the data are split, the TF-IDF vectorizer is fitted on the training text and applied to the test text, the Multinomial Naive Bayes model is fitted on the training vectors, and the model is evaluated on the

test vectors. Finally `joblib.dump()` writes `model.pkl` and `vectorizer.pkl`, and the metrics, sample counts and configuration are written to `metrics.json`.

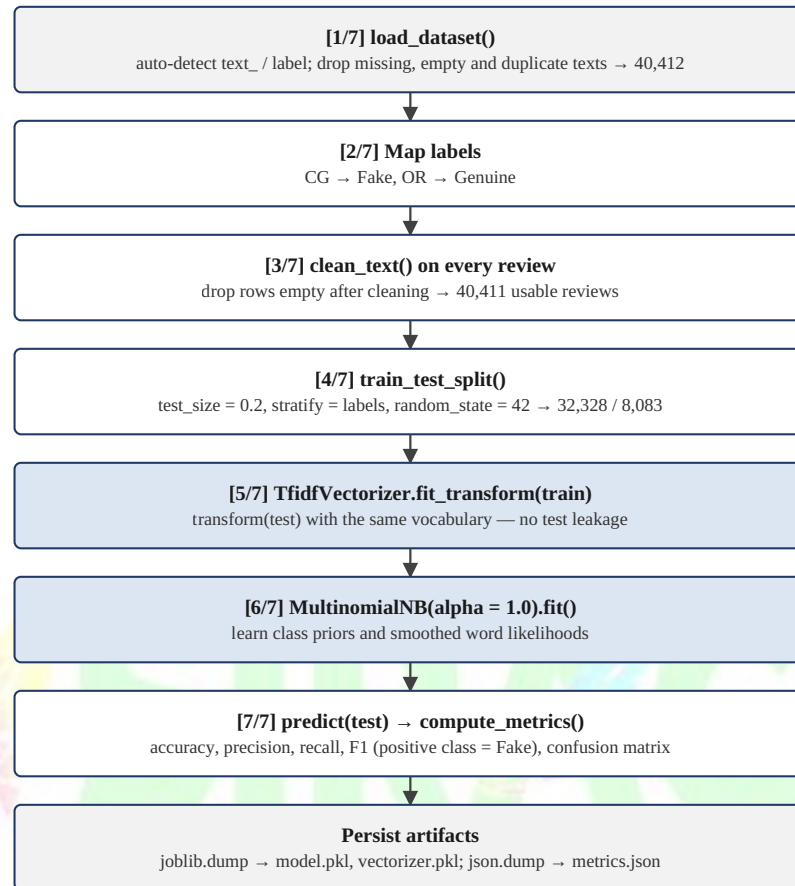


Figure 4.8 Model training and evaluation pipeline (`src/train_model.py`).

4.8 MODEL EVALUATION

The model is evaluated on the 8,083 test reviews by `compute_metrics()` in `src/evaluate.py`, with Fake as the positive class. Writing TP, TN, FP and FN for true positives, true negatives, false positives and false negatives [22]:

Table 4.5 Evaluation metrics

Metric	Definition	Question answered
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	What fraction of all test reviews is classified correctly?
Precision (Fake)	$TP / (TP + FP)$	When the model says “Fake”, how often is it right?
Recall (Fake)	$TP / (TP + FN)$	What fraction of the fake reviews does the model find?
F1-score (Fake)	$2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$	A single balance of precision and recall
Confusion matrix	Counts of actual versus predicted classes	Which kinds of error occur, and how often?

Because the classes are balanced, accuracy is a meaningful summary here; precision, recall and F1-score show whether the model favours one type of error over the other.

4.9 CONFUSION MATRIX

Figure 4.9 shows the confusion matrix of the model on the test set, taken from `models/metrics.json`. Of the 4,040 fake test reviews, 3,598 were classified as Fake and 442 as Genuine; of the 4,043 genuine test reviews, 3,626 were classified as Genuine and 417 as Fake. In total 7,224 reviews were classified correctly and 859 incorrectly.

		Predicted class	
		Fake	Genuine
Actual class	Fake	3,598 True Positive (TP)	442 False Negative (FN)
	Genuine	417 False Positive (FP)	3,626 True Negative (TN)

Figure 4.9 Confusion matrix on the 8,083-review test set (positive class = Fake).

From these counts, accuracy = $(3,598 + 3,626) / 8,083 = 89.37\%$, precision = $3,598 / (3,598 + 417) = 89.61\%$, recall = $3,598 / (3,598 + 442) = 89.06\%$ and F1-score = 89.34% . These results are discussed in Chapter 6.

CHAPTER 5: SYSTEM DESIGN AND IMPLEMENTATION

This chapter describes the architecture of the system, its workflow and modules, the technology stack, the use of the trained model in the prediction pipeline, and the implemented Streamlit application.

5.1 SYSTEM ARCHITECTURE

The system has two layers (Figure 5.1). The **presentation layer** is the Streamlit application in `app.py`, which renders the pages, keeps session state and collects the review text. The **ML inference layer** is the `src/` package; the interface calls one function, `predict_review()`, which preprocesses the text, applies the saved TF-IDF vectorizer and the saved Multinomial Naive Bayes model, and returns the prediction together with review insights. The trained artifacts are read from the `models/` folder once and cached.

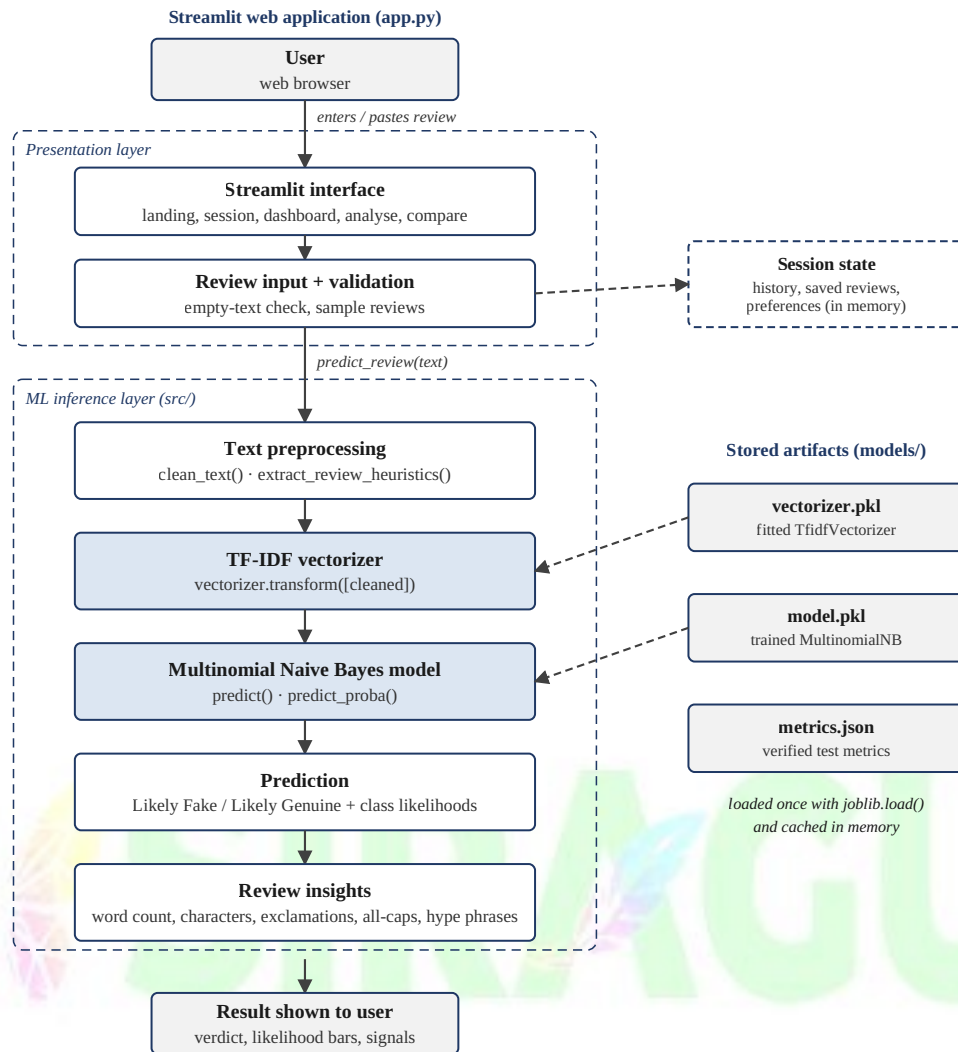


Figure 5.1 AI fake review detection system architecture.

5.2 OVERALL WORKFLOW

Figure 5.2 shows the application workflow from the user's point of view. After entering the application through the session login, the user opens the analyser and submits a review. The input is validated; if it is empty, a warning is shown and no prediction is made. Otherwise the text is preprocessed, the cached vectorizer and model are loaded, the text is transformed and classified, and the verdict, the class likelihoods and the supporting language signals are displayed. Each successful analysis is recorded in the session history, and the user can bookmark it, download a short text report, or analyse another review.

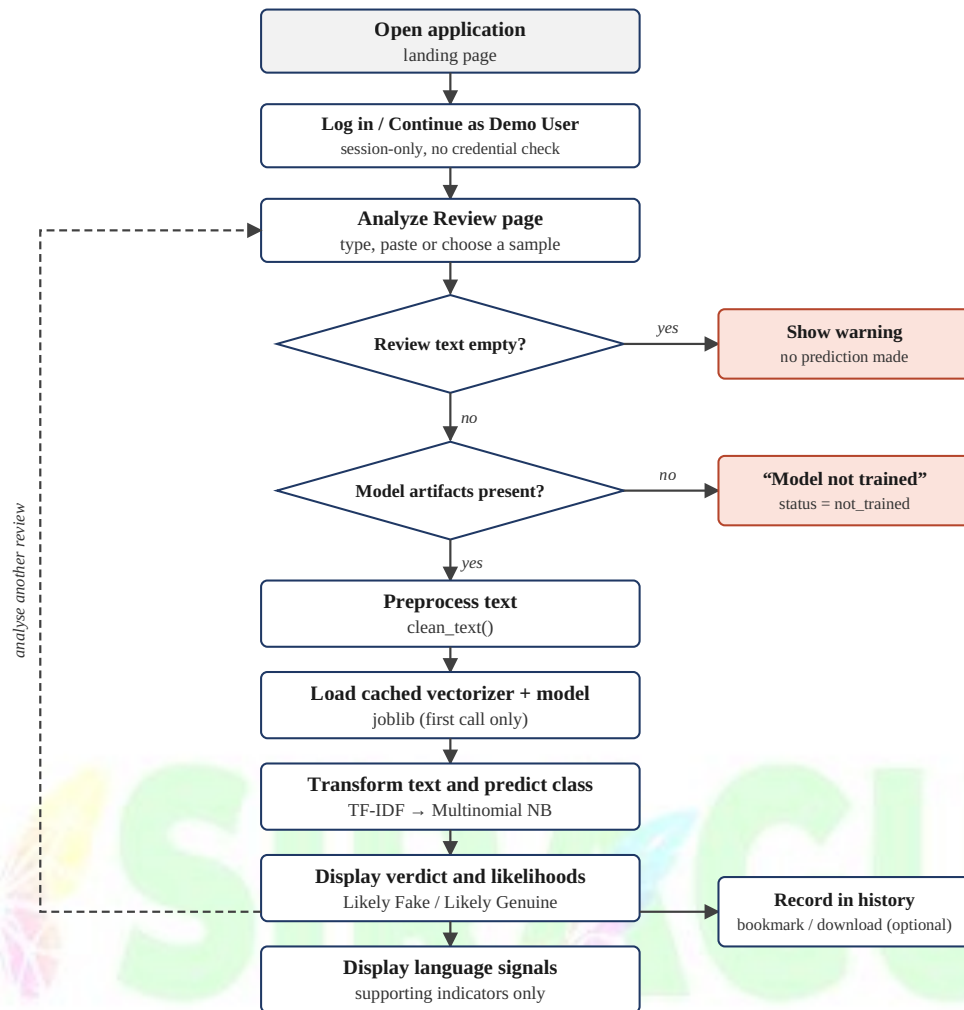


Figure 5.2 Overall application workflow.

5.3 NLP PIPELINE

The NLP pipeline used at prediction time is identical to the one used in training (Section 4.3): lowercase conversion, URL and HTML-tag removal, replacement of non-alphabetic characters, stopword and one-letter-token removal, and whitespace normalisation, followed by TF-IDF transformation with the stored vocabulary. Using the same function and the same fitted vectorizer guarantees that a review in the application is represented in the same 25,000-dimensional feature space as the training data.

5.4 MODULE DESCRIPTION

Table 5.1 Module description

Module	Responsibility
Data loading and validation	Detect text and label columns, report missing values, duplicates and label balance, load and de-duplicate data (<code>data_loader.py</code>)
Preprocessing	<code>clean_text()</code> for model input; <code>extract_review_heuristics()</code> for language signals (<code>preprocessing.py</code>)
Model training	Label mapping, split, TF-IDF fitting, Multinomial NB training, saving artifacts (<code>train_model.py</code>)
Evaluation	Accuracy, precision, recall, F1-score, confusion matrix and classification report (<code>evaluate.py</code>)
Prediction	Artifact checks and caching, preprocessing, vectorisation, prediction, probabilities, result dictionary (<code>predict.py</code>)
Streamlit application	Public pages, session login, dashboard, analyser, comparison, history, saved reviews, settings (<code>app.py</code>)

5.5 TECHNOLOGY STACK

Table 5.2 Technology stack

Technology	Purpose in the project
Python	Core development language for the ML pipeline and the application
Pandas	Reading the CSV dataset, validation statistics, de-duplication [15]
NumPy	Numerical arrays used by scikit-learn and the evaluation code [16]
NLTK	English stopword list for preprocessing [13]
Scikit-learn	<code>TfidfVectorizer</code> , <code>MultinomialNB</code> , <code>train_test_split</code> and evaluation metrics [11]
Joblib	Saving and loading <code>model.pkl</code> and <code>vectorizer.pkl</code> [17]
Streamlit	Web interface, page routing through session state, widgets [14]
VS Code / Antigravity	Development environment

Table 5.3 Project source files and responsibilities

File	Key functions / contents
src/data_loader.py	find_column_match(), inspect_dataset_file(), validate_dataset(), load_dataset()
src/preprocessing.py	clean_text(), extract_review_heuristics(), NLTK stopword loading with an offline fallback list
src/train_model.py	train(): seven-stage training pipeline and artifact saving
src/evaluate.py	compute_metrics(), format_metrics_summary()
src/predict.py	is_model_trained(), load_model_artifacts(), predict_review()
app.py	Page renderers (render_home() ... render_settings()), navigation bar, session state, router main()
models/metrics.json	Saved test metrics, confusion matrix, sample counts and configuration

5.6 MODEL IMPLEMENTATION

The trained model and vectorizer are stored as `models/model.pkl` (the fitted `MultinomialNB` object) and `models/vectorizer.pkl` (the fitted `TfidfVectorizer`, including its vocabulary and IDF weights). They are written once by the training pipeline with `joblib.dump()`. At run time, `is_model_trained()` confirms that both files exist, and `load_model_artifacts()` loads them with `joblib.load()` on the first prediction and keeps them in module-level variables, so subsequent predictions do not read the disk again. Because the vectorizer is stored together with the model, the application never refits it and cannot drift from the training representation.

The artifacts are tied to the scikit-learn version used to create them: they were saved with scikit-learn 1.7.2, and loading them with a newer version produces an `InconsistentVersionWarning`. During the preparation of this report the artifacts were re-loaded and re-evaluated on the reproduced test split, and gave the same confusion matrix as `metrics.json`; pinning the library version is nevertheless recommended (Section 6.8).

5.7 PREDICTION PIPELINE

A review entered in the application passes through `predict_review()` as shown in Figure 5.3:

1. If the artifacts are missing, the function returns `status = "not_trained"`; if the text is empty or whitespace, it returns `status = "empty_input"`.
2. `extract_review_heuristics()` computes the language signals from the raw text: character count, word count, number of “!” and “?” characters, number of all-capital words, and matches against a fixed list of promotional phrases.
3. `clean_text()` preprocesses the review exactly as in training.
4. `vectorizer.transform()` converts the cleaned text into a $1 \times 25,000$ TF-IDF vector.
5. `model.predict()` returns the class (CG or OR), mapped to Fake or Genuine; `model.predict_proba()` returns the probability of each class, converted to percentages.
6. The function returns a dictionary with the label (“Likely Fake Review” or “Likely Genuine Review”), the confidence (the larger class probability), both class likelihoods, the cleaned text and the heuristics.

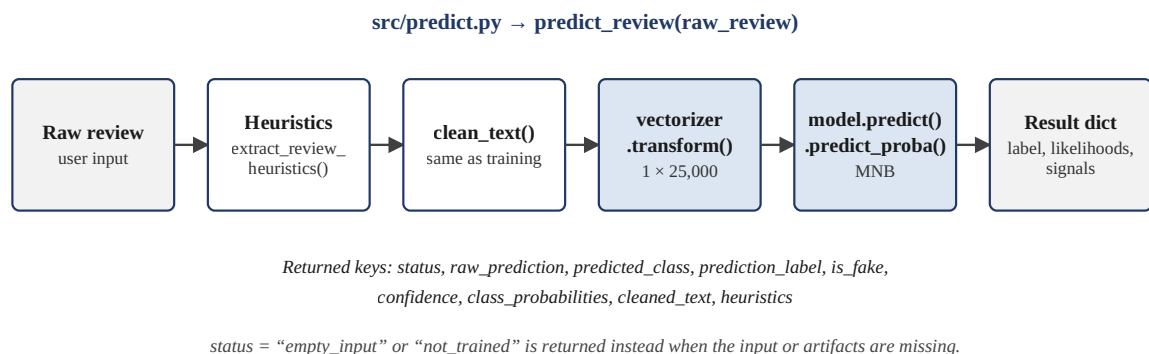


Figure 5.3 Model prediction pipeline (`predict_review()`).

The language signals are computed independently of the classifier and are not used as model features; the interface labels them as additional reading context that does not determine the classification.

5.8 STREAMLIT APPLICATION

The application is started with `streamlit run app.py` and served at `http://localhost:8501`. It is a single Streamlit script that uses `st.session_state` to store the current page, the signed-in user, the analysis history, the saved reviews, the active analysis and display preferences. A navigation bar of buttons changes `current_page`, and the router function `main()` renders the corresponding page. Internal pages are listed in `auth_routes`; if no user is signed in, the login page is shown instead. Custom CSS defines the card, banner and result styles.

Figures 5.4 to 5.13 were captured from the running application for this report (7 October 2026). The review texts in the figures are the sample reviews built into the analyser, and all likelihood values shown are the actual outputs of the saved model for those texts.

5.9 USER INTERFACE

Landing page. The public home page (Figure 5.4) introduces the problem, shows an illustrative mock-up of a result, offers a live “Try ReviewShield” demonstration with sample reviews, explains the four-step process, answers frequently asked questions and displays the verified test accuracy (89.37%), F1-score (0.8934) and dataset size (40,432). The mock-up card on the hero section is static illustration; live predictions are produced only by the demonstration and analyser controls. Separate Features, How It Works (Figure 5.5) and About pages describe the system and its limits.

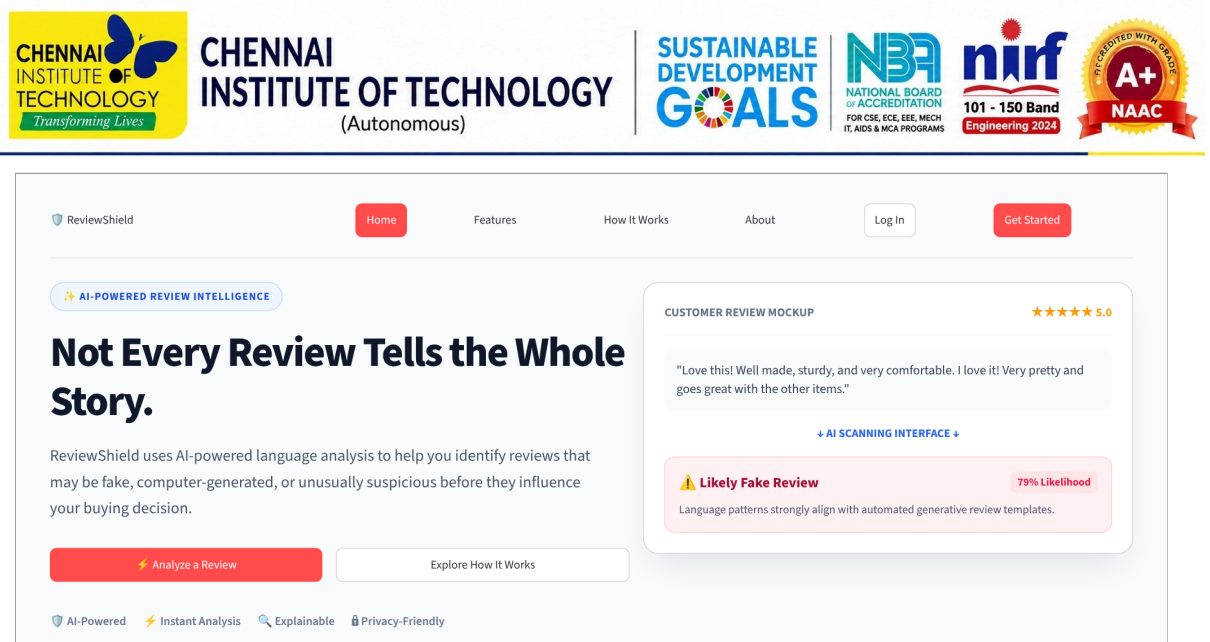


Figure 5.4 Streamlit landing page.

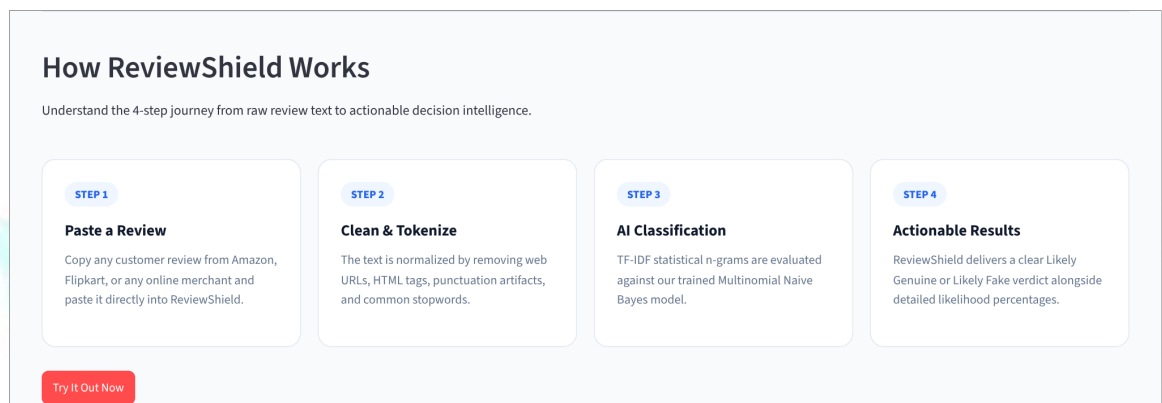


Figure 5.5 How It Works page describing the four-step analysis.

Login / session interface. The split-screen login page (Figure 5.6) has e-mail and password fields and a one-click “Continue as Demo User” option for evaluation; a Sign Up page collects a name and e-mail and checks that the two passwords match. These pages create a *session identity only*: any non-empty e-mail address is accepted, no password is stored or verified, and the session ends when the user logs out or closes the browser. Secure authentication is future work.

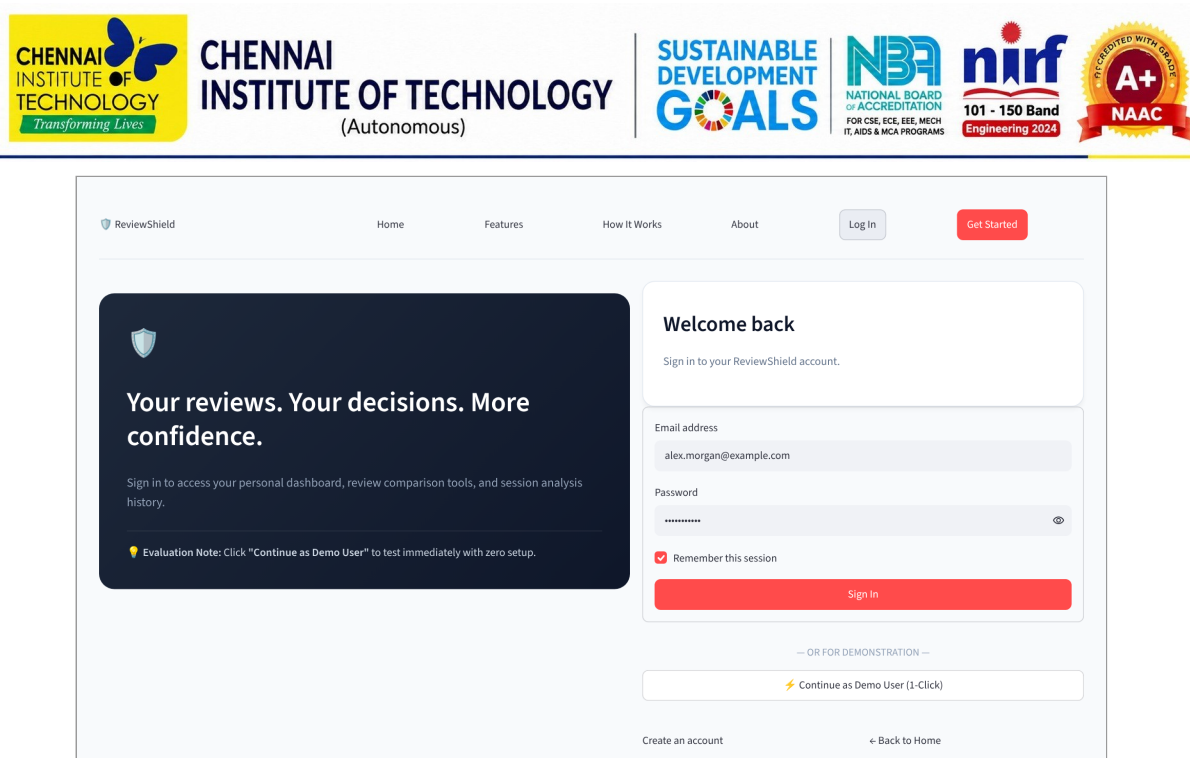


Figure 5.6 Login / session interface.

Dashboard. After login, the dashboard (Figure 5.7) greets the user, offers shortcuts to analyse or compare reviews, and shows session activity: the number of reviews analysed, how many were Likely Genuine and Likely Fake, the number of saved bookmarks, and the four most recent analyses with their verdicts and confidence. The figure shows the dashboard after the three sample reviews were analysed.

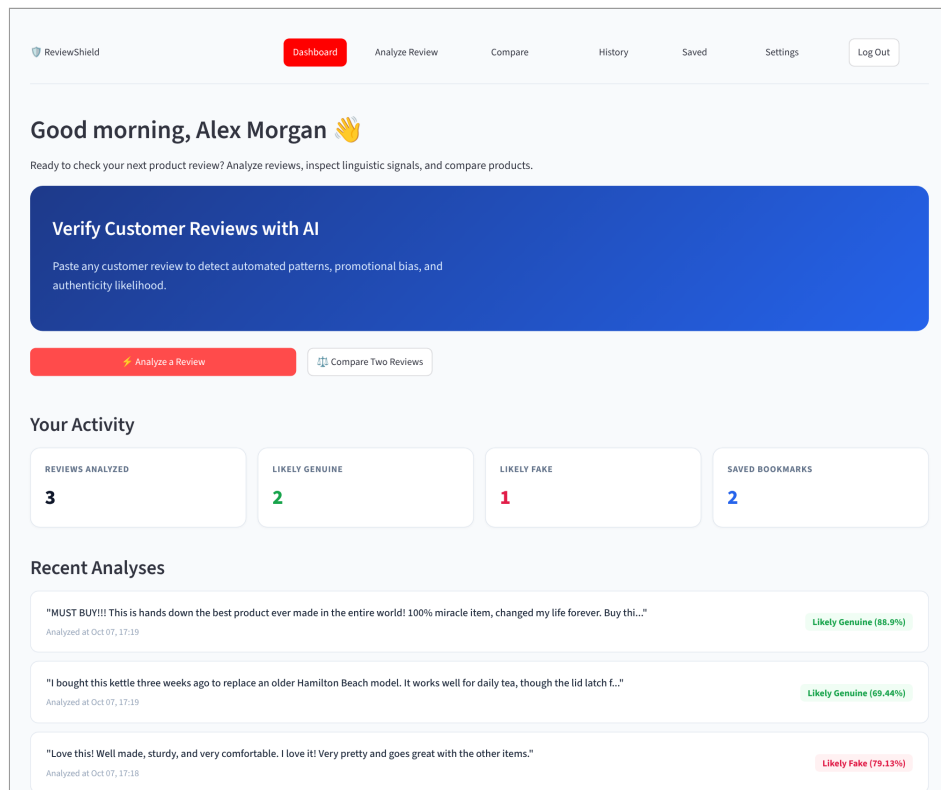


Figure 5.7 Dashboard with session activity and recent analyses.

Review analysis. The Analyze Review page (Figure 5.8) provides three quick samples (computer-generated style, authentic style and promotional style), a text area, a live word and character count, and Analyze and Clear buttons.

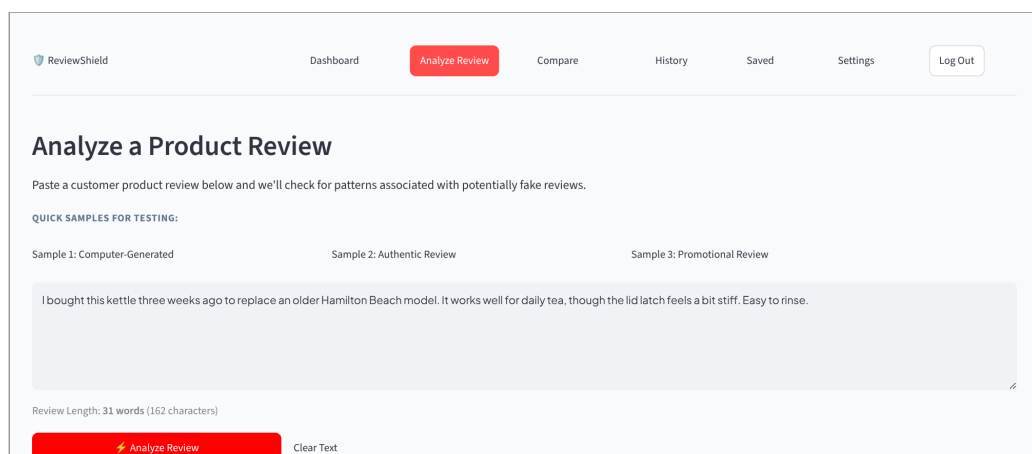


Figure 5.8 Review analysis interface.

Prediction result. The result (Figure 5.9) begins with a verdict banner — red for “Likely Fake Review”, green for “Likely Genuine Review” — followed by the Fake and Genuine likelihoods with progress bars, the language signals, a

short “Why this result?” explanation and actions to bookmark the review, download a plain-text report, or analyse another review. For the computer-generated-style sample *“Love this! Well made, sturdy, and very comfortable. I love it! Very pretty and goes great with the other items.”* the model returned Likely Fake with a Fake likelihood of 79.13% and a Genuine likelihood of 20.87%.

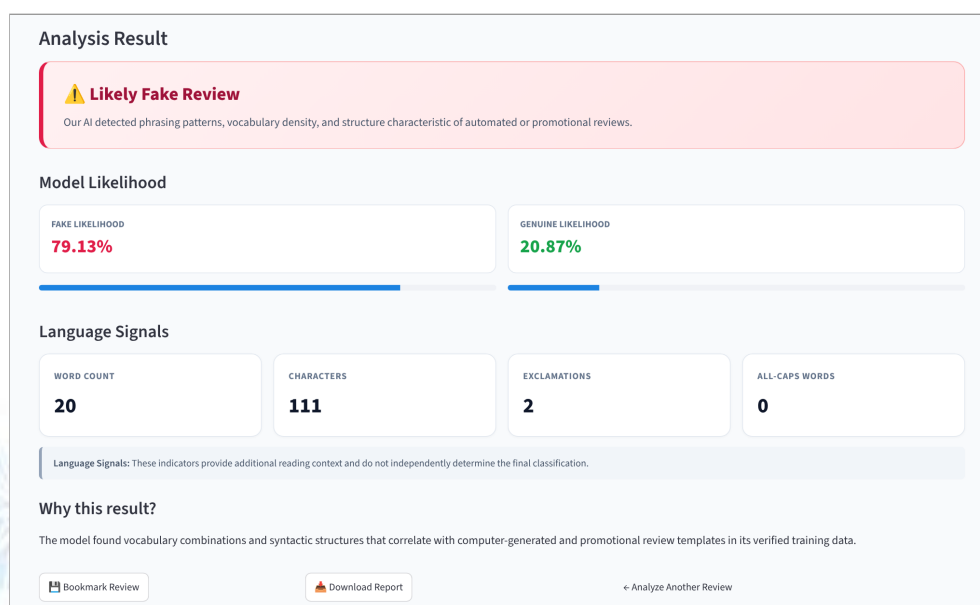


Figure 5.9 Prediction result for the computer-generated-style sample (Likely Fake).

Review insights. The Language Signals panel reports word count, characters, exclamation and question marks, all-capital words and any detected promotional phrases. Figure 5.10 shows the result for the promotional-style sample (“MUST BUY!!! This is hands down the best product ever made...”): the signals detect seven emphasis marks, two all-capital words and six promotional phrases, while the classifier returns Likely Genuine (88.9%). This case is discussed in Section 6.7; it illustrates why the interface keeps the signals separate from the model verdict. A dedicated insights view (`render_insights()`) exists in the source code but is not linked from the navigation bar in the current build, so the insights are presented within the analysis result.

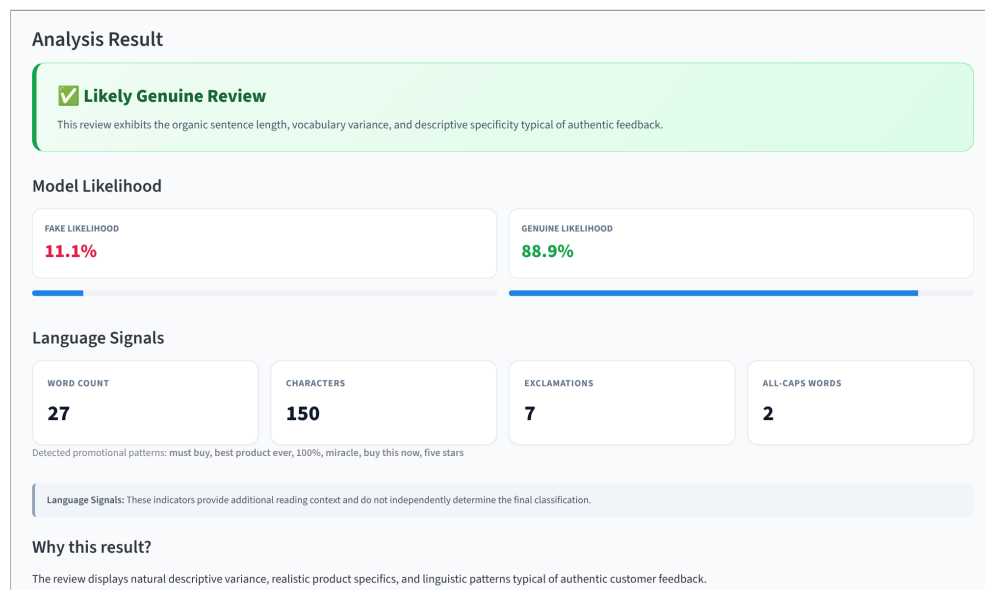


Figure 5.10 Prediction result with language signals for the promotional-style sample.

Comparison. The Compare page (Figure 5.11) analyses two reviews side by side and states which has the higher fake likelihood. For the two default texts, Review A was Likely Fake (79.13%) and Review B Likely Genuine (69.44%).

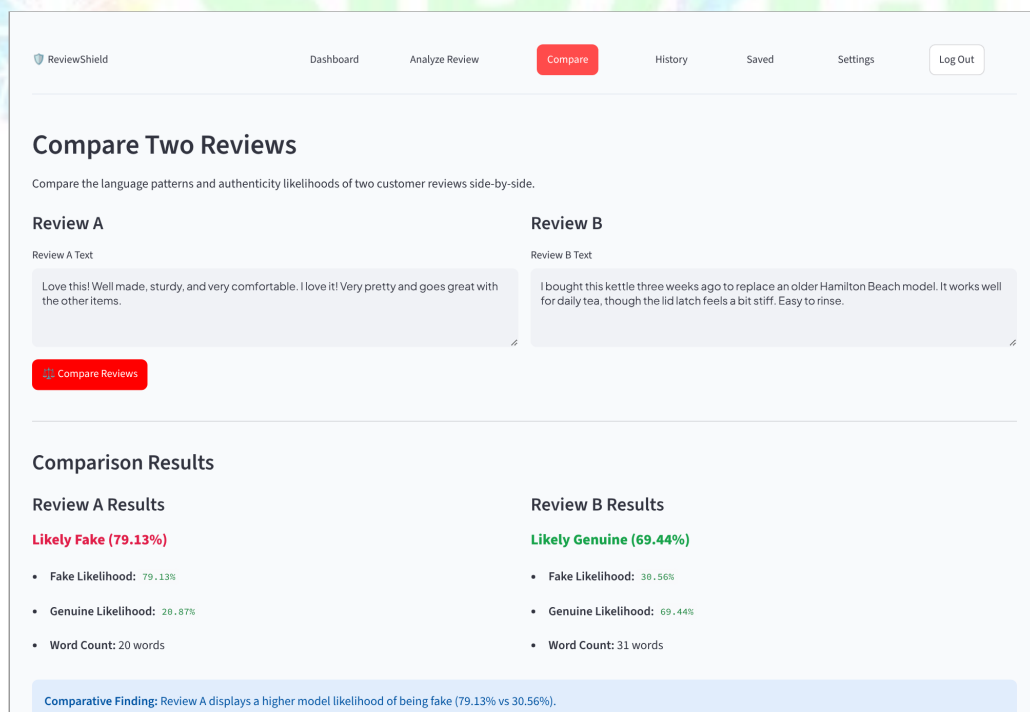


Figure 5.11 Review comparison interface.

History and saved reviews. The History page (Figure 5.12) lists every analysis of the session with its verdict, confidence and time, and supports

keyword search, a Likely Fake / Likely Genuine filter and clearing. The Saved page (Figure 5.13) lists reviews bookmarked from the analysis result.

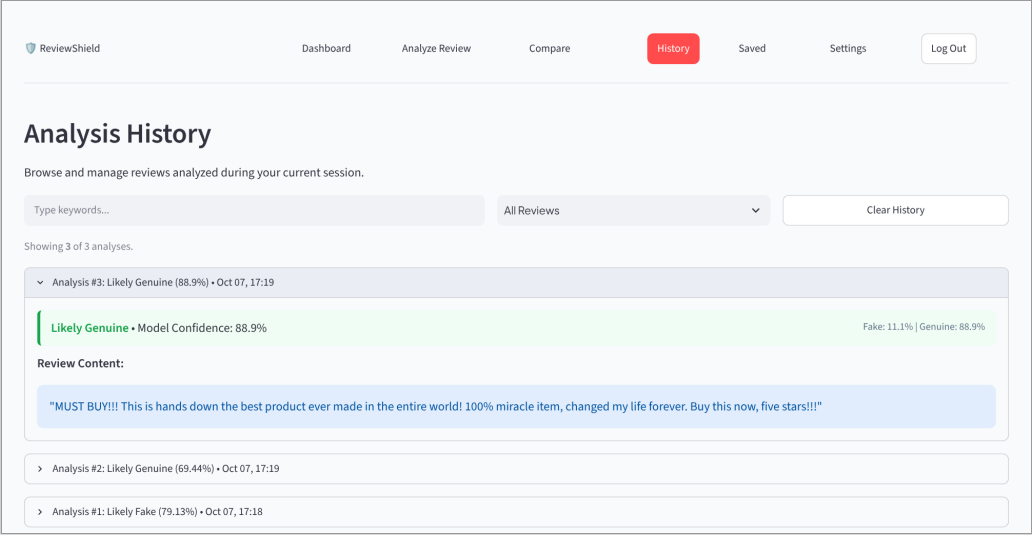


Figure 5.12 Session analysis history with search and filter.

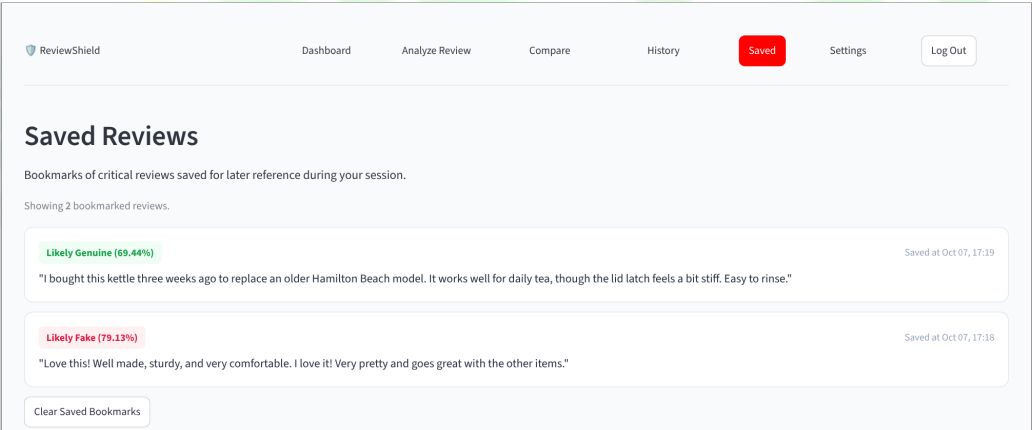


Figure 5.13 Saved (bookmarked) reviews.

Settings. The Settings page (Figure 5.14) shows the session account, display preferences, buttons to clear the history and bookmarks, an expandable panel with the model specification and verified metrics, and sign-out. The “Show Language Signals” preference controls the signals panel on the analysis result; the “Show Model Likelihood Probability Bars” preference is stored in the session but is not yet applied by the analysis page.

ReviewShield
Dashboard
Analyze Review
Compare
History
Saved
Settings
Log Out

Settings & Preferences

Configure your ReviewShield preferences and session options.

Profile Information

ACTIVE SESSION ACCOUNT
Alex Morgan
alex.morgan@example.com • Authenticated

Analysis Display Preferences

☐ Show Language Signals on Analysis Results
☒ Show Model Likelihood Probability Bars

Privacy & Session Data

AI Engine Specifications (Demonstration Reference)

Model Specifications:

- Classification Algorithm: Multinomial Naive Bayes (MNB) with Laplace smoothing
- Feature Extractor: TF-IDF Vectorizer (25,000 unigrams & bigrams)
- Verified Test Accuracy: 89.37% (Evaluated on 8,083 unseen samples)
- Verified Fake F1-Score: 0.8934 (Precision: 89.61%, Recall: 89.06%)

Figure 5.14 Settings interface with model specification panel.

5.10 PROJECT FOLDER STRUCTURE

Figure 5.15 shows the final folder structure of the project repository.

ReviewShield/	
├ app.py	Streamlit web application (all pages)
├ requirements.txt	Python dependencies
├ README.md	Project documentation
├ .gitignore	
├ dataset/	
│ └ fake reviews dataset.csv	40,432 labelled reviews
├ models/	
│ └ model.pkl	Trained Multinomial Naive Bayes model
│ └ vectorizer.pkl	Fitted TF-IDF vectorizer
│ └ metrics.json	Test-set metrics and configuration
├ src/	
│ └ __init__.py	
│ └ data_loader.py	Dataset inspection, validation and loading
│ └ preprocessing.py	clean_text() and language-signal heuristics
│ └ train_model.py	TF-IDF + Multinomial NB training pipeline
│ └ evaluate.py	Accuracy, precision, recall, F1, confusion matrix
│ └ predict.py	Inference: load artifacts, predict_review()
└ docs/report/	This report, its figures and build scripts

Figure 5.15 Project folder structure.

CHAPTER 6: RESULTS AND DISCUSSION

This chapter reports the evaluation of the model on unseen data, the testing of the application, and what the results do and do not show.

6.1 EVALUATION METHODOLOGY

The model was evaluated once on the 8,083-review test set created by the stratified 80/20 split in Section 4.4. None of these reviews was used to fit the TF-IDF vocabulary, the IDF weights or the Naive Bayes parameters, and duplicate texts were removed before the split, so the test set measures performance on reviews the model has not seen. Metrics were computed with scikit-learn using Fake as the positive class and saved to `models/metrics.json`. While preparing this report, the saved artifacts were re-loaded and applied to the reproduced test split, and the same values were obtained.

6.2 MODEL PERFORMANCE

Table 6.1 Model performance on the test set

Metric	Value
Accuracy	89.37%
Precision (Fake)	89.61%
Recall (Fake)	89.06%
F1-Score (Fake)	89.34%

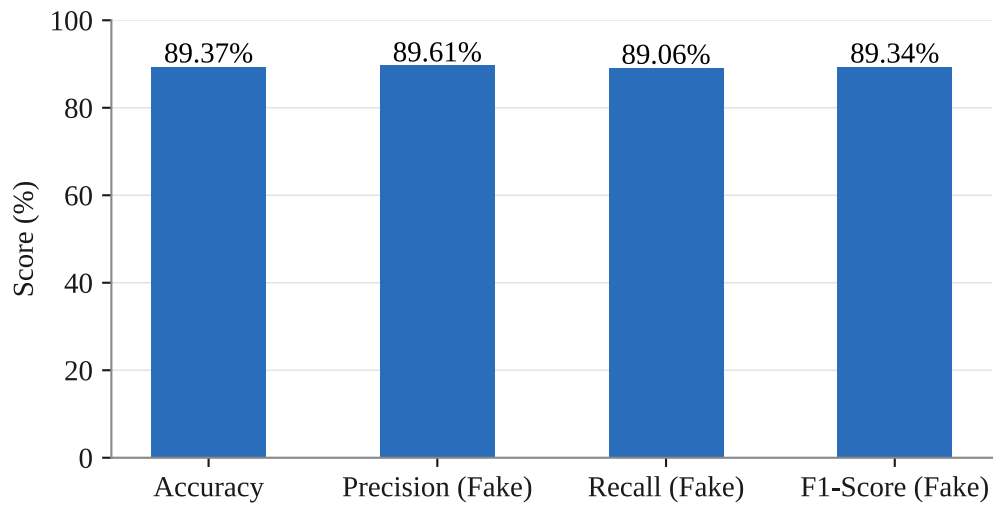


Figure 6.1 Model performance on the 8,083-review test set.

The classification report in `metrics.json` also gives the per-class values (Figure 6.2): for the Genuine class the precision is 89.13%, the recall 89.69% and the F1-score 89.41%. The macro-averaged F1-score is 89.37%. The two classes are recognised almost equally well.

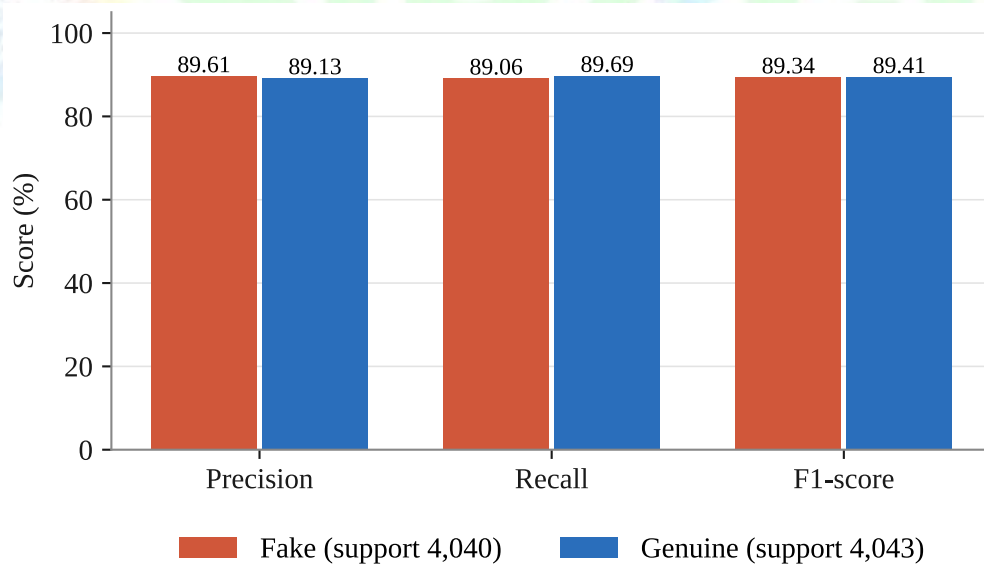


Figure 6.2 Precision, recall and F1-score per class (from the saved classification report).

6.3 CONFUSION MATRIX

Table 6.2 Confusion matrix values

	Predicted Fake	Predicted Genuine	Total
Actual Fake	3,598 (TP)	442 (FN)	4,040
Actual Genuine	417 (FP)	3,626 (TN)	4,043
Total	4,015	4,068	8,083

The matrix (also Figure 4.9) shows 7,224 correct and 859 incorrect classifications. The two error types are of similar size: 442 fake reviews were classified as genuine and 417 genuine reviews as fake.

6.4 CLASSIFICATION RESULTS

The results show that, on this dataset, word and two-word phrase statistics separate computer-generated from original reviews for roughly nine reviews in ten. A precision of 89.61% means that about one in ten reviews flagged as fake is in fact genuine; a recall of 89.06% means that about one in nine fake reviews is not flagged. The near-equal precision and recall, and the near-equal results for both classes, indicate that the model is not biased towards either label, which is consistent with the balanced training data.

These figures describe performance on test data drawn from the same dataset as the training data. They should not be read as the accuracy the system would achieve on arbitrary reviews from other websites, languages, product types or generation methods; performance is dataset-dependent, and an accuracy of 89.37% does not make any individual prediction certain.

6.5 APPLICATION TESTING

The application was tested manually against the cases in Table 6.3. Cases marked **Observed** were carried out on the running application while preparing this report and are documented by the figures cited; cases marked **Designed**

are test cases derived from the code whose expected behaviour follows from the implementation but for which no recorded execution is reported.

Table 6.3 Application test cases

ID	Test case	Input / action	Expected behaviour	Result
TC01	Model loading	Start application with artifacts present	Pages load; predictions available	Observed (Fig. 5.4)
TC02	Valid review input	Sample 1 (computer-generated style)	Verdict, likelihoods and signals shown	Observed: Likely Fake, 79.13% (Fig. 5.9)
TC03	Valid review input	Sample 2 (authentic style)	Verdict, likelihoods and signals shown	Observed: Likely Genuine, 69.44% (Fig. 5.7)
TC04	Review with punctuation / capitals	Sample 3 (promotional style)	Signals report emphasis and phrases	Observed: 7 marks, 2 all-caps words, 6 phrases (Fig. 5.10)
TC05	Empty review	Click Analyze with empty text	Warning; no prediction or history entry	Observed
TC06	Prediction generation	Compare two default reviews	Both analysed; higher fake likelihood named	Observed (Fig. 5.11)
TC07	Navigation	Use navigation bar on all public and internal pages	Correct page rendered	Observed (Figs. 5.4–5.14)
TC08	Session / login interface	Continue as Demo User	Dashboard opens with session user	Observed (Fig. 5.7)
TC09	History and bookmarks	Analyse three reviews; bookmark two	Counts 3 / 2 / 1 / 2 on dashboard	Observed (Figs. 5.7, 5.12, 5.13)
TC10	Short review	A one- or two-word review	Prediction returned; low word count shown	Designed
TC11	Long review	A review of several paragraphs	Prediction returned; signals updated	Designed
TC12	Review containing URLs	Text with <code>http://</code> or <code>www.</code> link	URL removed by <code>clean_text()</code> before vectorisation	Designed (behaviour of <code>clean_text()</code> shown in Section 4.3)
TC13	Punctuation-only review	e.g. “!!!”	Fallback to lower-cased raw text; prediction returned	Designed
TC14	Missing model artifacts	Remove <code>models/</code> *.pkl	“Model not trained” status, no crash	Designed

ID	Test case	Input / action	Expected behaviour	Result
TC15	Protected routes	Open an internal page without a session user	Login page shown instead	Designed

Table 6.4 Test results summary

Category	Test cases	Observed	Designed (not executed in this report)
Model and prediction	TC01–TC04, TC06	5	0
Input validation	TC05, TC10–TC13	1	4
Navigation and session	TC07–TC09, TC15	3	1
Failure handling	TC14	0	1
Total	15	9	6

No automated unit-test suite is included in the repository; the model metrics are the result of the scripted evaluation, and the interface checks above were performed by hand. Automated tests are listed as future work.

6.6 DISCUSSION

The evaluation answers the driving question within its stated limits. A lightweight pipeline — seven preprocessing steps, a 25,000-feature TF-IDF representation and a Naive Bayes classifier with a single smoothing parameter — classifies 89.37% of unseen reviews correctly, trains in seconds on a CPU, and runs fast enough to give interactive results in a web application. Because it is probabilistic, the application can present a likelihood for both classes instead of a bare verdict, and the separate language signals give the reader concrete features of the text to examine.

The value of stratified splitting, de-duplication and fitting the vectorizer on training data only is that the reported numbers are an honest estimate for data from the same source. Without these steps, identical reviews in both sets or a vocabulary built from the test set would have inflated the results.

6.7 ERROR ANALYSIS

False negatives (442). These are computer-generated reviews that the model accepted as genuine, 10.9% of the 4,040 fake test reviews. A generated review that uses the same vocabulary and phrases as typical original reviews offers few distinguishing n-grams, and a bag-of-n-grams model cannot use word order beyond adjacent pairs or the overall coherence of the text.

False positives (417). These are genuine reviews flagged as fake, 10.3% of the 4,043 genuine test reviews. Short, generic praise written by real customers (“great product, works well”) shares its vocabulary with the generated reviews, and the model has no information about the reviewer that could resolve the ambiguity. In a moderation setting, a false positive is the more costly error for an honest customer, which is why the application uses the wording “Likely” rather than a definitive label.

What the model has learned. The CG class in the training data consists of computer-generated reviews, not of human-written promotional spam. The promotional sample in Figure 5.10 — written by a person in an exaggerated style, with capitals, exclamation marks and phrases such as “must buy” — was classified as Likely Genuine (88.9%). After preprocessing removes punctuation and capitalisation, its remaining words resemble enthusiastic original reviews more than the generated ones. This is not a malfunction but a direct consequence of the training labels: the model detects the kind of fakeness present in its dataset. It also shows why the language signals are reported separately, and why a single text-only classifier should not be treated as a general fraud detector.

6.8 LIMITATIONS

- The model depends on its training data: it recognises the computer-generated reviews of this dataset and may not generalise to other generation methods or to human-written fake reviews.
- Performance is language- and domain-dependent; the project primarily targets English product reviews in the ten categories of the dataset.
- Classification is text-only and does not independently verify a reviewer, a purchase, timing or rating behaviour.
- A “Likely Fake” prediction is a statistical indication, not proof that a review is fraudulent.
- Advanced transformer models such as BERT or RoBERTa are not part of the current model, and no other classifier was trained for comparison.
- The login is session-only, with no credential verification or persistent storage; one stored display preference is not yet applied; the insights view is not linked in the navigation.
- The saved artifacts depend on the scikit-learn version used to create them (1.7.2); `requirements.txt` specifies minimum versions only.
- Application testing was manual; no automated test suite or load test is included.

6.9 PRACTICAL INTERPRETATION

For a customer, the prototype is a second opinion: pasting a suspicious review shows whether its wording resembles computer-generated text and which surface features (length, emphasis, promotional phrases) stand out, which can prompt a closer reading before a purchase. For an e-commerce platform, a classifier of this kind could serve as one inexpensive first-stage filter that prioritises reviews for human moderators, combined with behavioural and account-level signals that a platform holds but an individual customer does not. In both roles the output should support a human decision, not replace it.

CHAPTER 7: TEAM REFLECTION AND LEARNING OUTCOMES

7.1 INDIVIDUAL REFLECTIONS

Santhosh (2104251040010): My work centred on the machine-learning side of the project. Writing the dataset loader taught me to check data before trusting it: the validation step showed the duplicate rows and duplicate texts, and I learned why duplicate texts must be removed before splitting, otherwise the same review could be both trained on and tested on. Implementing the preprocessing function and then importing the same function in the prediction module made clear how easily training and inference can drift apart. Configuring the TF-IDF vectorizer helped me understand the difference between a feature extraction technique and a classifier, and why the vectorizer must be fitted only on training data. Working through Bayes' theorem, class priors and Laplace smoothing for Multinomial Naive Bayes, and computing precision, recall and F1 from the confusion matrix by hand, gave me a much better feel for what the reported numbers mean and where they stop applying.

Aathithiyan (2104251040020): I was responsible for the Streamlit application and its integration with the model. Building a multi-page interface in Streamlit taught me how its re-run model works and how `st.session_state` can hold the current page, the signed-in user, history and bookmarks. Designing the result screen required decisions about presentation: showing likelihoods for both classes, using the word "Likely" in every verdict, and labelling the language signals as supporting context so that users do not mistake them for the model's reasoning. Testing the sample reviews — in particular the promotional review that the model classified as genuine — showed me that an interface must explain the limits of a model, not only its results. Debugging the connection between the interface and `predict_review()` also taught me to rely on the

status values returned by the backend instead of assuming that a prediction is always available.

7.2 TEAM LEARNING

As a team we learned that most of the effort in an ML project lies outside the algorithm: in validating the data, keeping preprocessing consistent, evaluating honestly and communicating uncertainty. Agreeing early on the contract between the two parts of the system — a single `predict_review()` function returning a dictionary with fixed keys — let the model and the interface be developed in parallel and integrated with few conflicts. Preparing the report and poster required us to separate what had been implemented and measured from what was only planned, and to describe both accurately.

7.3 TECHNICAL SKILLS ACQUIRED

Table 7.1 Technical skills and supporting evidence

Skill area	Evidence in the project
Python programming	Modular <code>src/</code> package; type-hinted functions; command-line entry points
Dataset validation	<code>validate_dataset()</code> : schema detection, missing values, duplicates, label balance
NLP preprocessing	<code>clean_text()</code> with regular expressions and NLTK stopwords
Feature extraction (TF-IDF)	Unigram + bigram vectorizer, <code>min_df</code> , 25,000 features, fit on training data only
Classification (Naive Bayes)	<code>MultinomialNB(alpha=1.0)</code> ; probabilities via <code>predict_proba()</code>
Model evaluation	Accuracy, precision, recall, F1-score, confusion matrix; error analysis
Model persistence	Joblib artifacts; version-compatibility awareness
Streamlit and UI/UX	Multi-page application with session state, result cards, comparison, history
Debugging and testing	Status handling for empty input and missing artifacts; manual test cases
Technical documentation	README, this report, diagrams and poster

7.4 COURSE OUTCOMES / EVIDENCE SUMMARY

Table 7.2 Learning outcomes and evidence

Learning area	Evidence	Section
Problem analysis and requirements	Driving question, objectives, requirement tables	Ch. 1, 3
Literature and concept exploration	Survey of approaches; comparison table	Ch. 2
Data handling and ML methodology	Validation, preprocessing, TF-IDF, Naive Bayes, evaluation	Ch. 4
System design and implementation	Architecture, modules, prediction pipeline, application	Ch. 5
Evaluation and critical analysis	Metrics, confusion matrix, error analysis, limitations	Ch. 6
Teamwork, communication and reflection	Roles, progress log, reflections, report and poster	Ch. 3, 7



CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

This project developed an end-to-end NLP machine-learning pipeline for fake product review detection. A labelled dataset of 40,432 reviews, balanced between computer-generated and original reviews, was validated and de-duplicated; each review was preprocessed with a single, shared cleaning function; TF-IDF was used as the feature extraction technique with 25,000 unigram and bigram features; and a Multinomial Naive Bayes classifier with Laplace smoothing was trained as the classification algorithm. On 8,083 unseen test reviews the model achieved **89.37% accuracy**, with a precision of 89.61%, a recall of 89.06% and an F1-score of 89.34% for the fake class.

The trained model and vectorizer are served through a Streamlit application that lets a user analyse a review in real time, see the likelihood of each class with supporting language signals, compare two reviews, and keep a session history and bookmarks. The project therefore answers its driving question: a lightweight and explainable text classifier, presented through an accessible web interface, can identify reviews whose language resembles computer-generated reviews. It also shows the limits of such a classifier — its dependence on the training data, its text-only view and its probabilistic nature — which the application and this report state explicitly.

8.2 FUTURE SCOPE

The following items are **not implemented** in the current system and are proposed as future work:

- Training and evaluating on larger and more diverse datasets, including human-written fake reviews and reviews from other platforms.
- Multilingual review detection, including Indian languages.

- Product URL and product-page analysis, with product-level aggregation of review predictions.
- Systematic comparison with Logistic Regression and Support Vector Machines on the same split.
- Deep-learning approaches and fine-tuned transformer models such as BERT and RoBERTa.
- Explainable AI features, such as showing the n-grams that contributed most to each prediction.
- Integration as a first-stage filter in large-scale e-commerce moderation, combined with behavioural signals.
- A REST API so that other applications can request predictions.
- Secure authentication with persistent user storage, pinned library versions, automated tests and a hosted deployment.



REFERENCES

- [1] J. Salminen, C. Kandpal, A. M. Kamel, S.-G. Jung, and B. J. Jansen, "Creating and detecting fake reviews of online products," *Journal of Retailing and Consumer Services*, vol. 64, Art. no. 102771, 2022.
- [2] M. Ott, Y. Choi, C. Cardie, and J. T. Hancock, "Finding deceptive opinion spam by any stretch of the imagination," in *Proc. 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, Portland, OR, USA, 2011, pp. 309–319.
- [3] N. Jindal and B. Liu, "Opinion spam and analysis," in *Proc. International Conference on Web Search and Data Mining (WSDM)*, 2008, pp. 219–230.
- [4] M. Luca and G. Zervas, "Fake it till you make it: Reputation, competition, and Yelp review fraud," *Management Science*, vol. 62, no. 12, pp. 3412–3427, 2016.
- [5] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/>
- [6] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA, USA: O'Reilly Media, 2009.
- [7] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [8] K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval," *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.
- [9] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [10] A. McCallum and K. Nigam, "A comparison of event models for naive Bayes text classification," in *Proc. AAAI-98 Workshop on Learning for Text Categorization*, 1998, pp. 41–48.
- [11] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [12] scikit-learn developers, "sklearn.feature_extraction.text.TfidfVectorizer" and "Naive Bayes," scikit-learn User Guide. [Online]. Available: https://scikit-learn.org/stable/modules/naive_bayes.html
- [13] NLTK Project, "Natural Language Toolkit (NLTK) Documentation." [Online]. Available: <https://www.nltk.org/>
- [14] Streamlit Inc., "Streamlit Documentation." [Online]. Available: <https://docs.streamlit.io/>
- [15] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conference (SciPy)*, 2010, pp. 56–61.
- [16] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

- [17] Joblib developers, “Joblib: running Python functions as pipeline jobs.” [Online]. Available: <https://joblib.readthedocs.io/>
- [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [19] Y. Liu *et al.*, “RoBERTa: A robustly optimized BERT pretraining approach,” arXiv: 1907.11692, 2019.
- [20] T. Joachims, “Text categorization with support vector machines: Learning with many relevant features,” in *Proc. 10th European Conference on Machine Learning (ECML)*, LNCS vol. 1398, 1998, pp. 137–142.
- [21] F. Sebastiani, “Machine learning in automated text categorization,” *ACM Computing Surveys*, vol. 34, no. 1, pp. 1–47, 2002.
- [22] D. M. W. Powers, “Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation,” *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011.



APPENDIX

A.1 PROJECT FOLDER STRUCTURE

The repository layout is shown in Figure 5.15. The commands used to validate the dataset, train the model and start the application are:

```
pip install -r requirements.txt
python -m src.data_loader          # validate the dataset
python -m src.train_model          # train, evaluate and save model.pkl, vectorizer.pkl,
metrics.json                      # metrics
streamlit run app.py              # start the web application (http://localhost:8501)
```

A.2 IMPORTANT SOURCE CODE

The excerpts below are taken from the project source. Print statements are omitted, “...” marks omitted lines, and end-of-line comments were added for this report where the caption says so.

1. Preprocessing function (src/preprocessing.py; end-of-line comments added)

```
def clean_text(text: str) -> str:
    if text is None or not isinstance(text, str):
        return ""
    text = text.lower()                    # lowercase
    text = re.sub(r'https?://\S+|www\.\S+', ' ', text)    # URLs
    text = re.sub(r'<.*?>', ' ', text)                # HTML tags
    text = re.sub(r'[^a-zA-Z\s]', ' ', text)            # punctuation, symbols, digits
    tokens = text.split()
    filtered_tokens = [word for word in tokens if word not in STOPWORDS and len(word) > 1]
    return " ".join(filtered_tokens)                # normalised whitespace
```

2. TF-IDF configuration and Multinomial Naive Bayes training (src/train_model.py; comments added)

```
X_train, X_test, y_train, y_test = train_test_split(
    cleaned_texts, labels, test_size=test_size, random_state=random_state, stratify=labels)

vectorizer = TfidfVectorizer(ngram_range=ngram_range, min_df=min_df, max_features=max_features)
X_train_vec = vectorizer.fit_transform(X_train)    # fit on training data only
X_test_vec = vectorizer.transform(X_test)          # same vocabulary and IDF weights

model = MultinomialNB(alpha=1.0)                  # Laplace smoothing
model.fit(X_train_vec, y_train)

y_pred = model.predict(X_test_vec)
metrics = compute_metrics(y_test, y_pred, pos_label="Fake")
joblib.dump(model, MODEL_PATH)
joblib.dump(vectorizer, VECTORIZER_PATH)
```

3. Prediction pipeline (src/predict.py, abridged)

```
heuristics = extract_review_heuristics(raw_review)
cleaned = clean_text(raw_review)
if not cleaned:
    # Fallback if text contained only punctuation or stopwords
    cleaned = raw_review.lower().strip()
model, vectorizer = load_model_artifacts()
...
text_vectorized = vectorizer.transform([cleaned])
raw_pred = model.predict(text_vectorized)[0]
raw_pred_str = str(raw_pred).strip()
standard_class = LABEL_MAPPING.get(raw_pred_str, raw_pred_str)
...
proba = model.predict_proba(text_vectorized)[0]
for cls_name, prob in zip(model.classes_, proba):
    mapped_cls = LABEL_MAPPING.get(str(cls_name), str(cls_name))
    class_probabilities[mapped_cls] = round(float(prob) * 100, 2)
confidence = round(float(max(proba)) * 100, 2)
```

A.3 MODEL TRAINING CONFIGURATION

Item	Value
Dataset	dataset/fake reviews dataset.csv; text column text_; label column label
Rows	40,432 loaded; 40,412 after removing duplicate texts; 40,411 after preprocessing
Label mapping	CG → Fake; OR → Genuine
Train / test split	80% / 20%, stratified, random_state = 42; 32,328 / 8,083 reviews
TF-IDF parameters	ngram_range = (1, 2), min_df = 3, max_features = 25000; 25,000 features
Classifier	MultinomialNB(alpha = 1.0)
Evaluation metrics	Accuracy 0.8937; Precision (Fake) 0.8961; Recall (Fake) 0.8906; F1 (Fake) 0.8934; confusion matrix [[3598, 442], [417, 3626]]
Saved artifacts	models/model.pkl, models/vectorizer.pkl, models/metrics.json

A.4 SAMPLE PREDICTION WORKFLOW

The workflow below traces an illustrative review through the pipeline. The cleaned text is the actual output of `clean_text()`; no probability is stated for this example. Measured outputs for the application's sample reviews are given in Section 5.9.

```
Review text    : "This product is AMAZING!!! Best purchase ever, visit www.deals-shop.com
                  <br> 10/10 would buy again :)"          (illustrative input)
|
v clean_text()
Cleaned text   : "product amazing best purchase ever visit would buy"
|
v vectorizer.transform([cleaned])
TF-IDF vector : 1 x 25,000 sparse vector; non-zero weights only for those unigrams and
                  bigrams of the cleaned text that are in the fitted vocabulary
|
v model.predict() / model.predict_proba()    (Multinomial Naive Bayes)
Prediction     : "CG" -> Likely Fake Review  or  "OR" -> Likely Genuine Review,
                  with the probability of each class reported as a percentage
```



NATURAL LANGUAGE PROCESSING • MACHINE LEARNING
**AI-POWERED FAKE PRODUCT REVIEW
DETECTION USING NLP**Classifying customer reviews as **Likely Fake** or **Likely Genuine** with TF-IDF + Multinomial Naive BayesPROJECT TEAM
Santhosh
Register No. 2104251040010PROJECT TEAM
Adithyan
Register No. 2104251040020FACULTY GUIDE
Mr. Pratham Verma, M.Tech.
Assistant Professor**01 ABSTRACT**

Online product reviews strongly influence customer purchasing decisions, but fake and computer-generated reviews can mislead consumers and reduce trust in e-commerce platforms. This project develops an NLP-based machine learning system to classify product reviews as **Likely Fake** or **Likely Genuine**. The system preprocesses review text, extracts TF-IDF features, and applies a Multinomial Naive Bayes classifier. A Streamlit-based application provides real-time review analysis and supporting linguistic insights.

02 PROBLEM STATEMENT

- Online shoppers increasingly rely on customer reviews before purchasing.
- Fake or computer-generated reviews can artificially influence customer decisions.
- Highly promotional, repetitive, or suspicious review language can reduce trust.
- Manual identification of fake reviews is difficult and time-consuming.
- A simple AI-assisted review analysis system can provide additional decision support.

**03 OBJECTIVES**

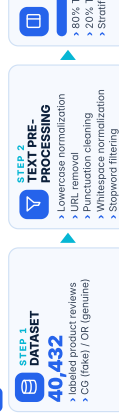
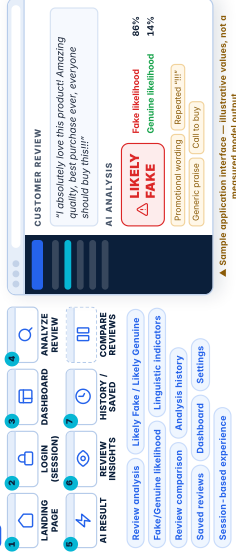
- Develop an NLP-based fake product review classification system.
- Preprocess customer review text consistently for training and prediction.
- Extract meaningful textual features using TF-IDF.
- Classify reviews using Multinomial Naive Bayes.
- Provide a user-friendly Streamlit application for real-time review analysis.

04 EXISTING SYSTEM

- Manual user reporting of suspicious reviews.
 - Basic keyword or blacklist filtering.
 - Account-level frequency checks.
 - Limited understanding of review language and context.
 - Difficulty identifying sophisticated or computer-generated reviews.
- Limitations: rule-based approaches may miss complex review patterns.

05 PROPOSED SYSTEM

An NLP and machine-learning based system that analyzes the language patterns of customer reviews and classifies them as **Likely Fake** or **Likely Genuine**.

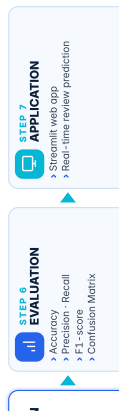
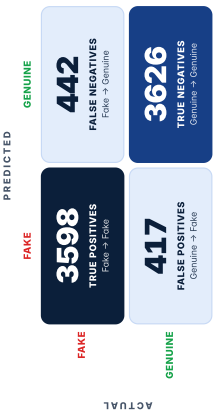
**06 METHODOLOGY****07 NLP PIPELINE****08 SYSTEM ARCHITECTURE****09 IMPLEMENTATION & APPLICATION OUTPUT****10 REFERENCES**

- J. Samminen, C. Kandpal, A. M. Kamel, S.-G. Jung, B. U. Jansen, "Creating and detecting fake reviews of online products," *J. Retailing and Consumer Services*, vol. 84, 102771, 2022.
- S. Bird, E. Klein, E. Loper, *Natural Language Processing with Python*, O'Reilly Media, 2009.

- F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, pp. 2825-2830, 2011.
- C. D. Manning, P. Ng, H. Schütze, *Introduction to Information Retrieval*, Cambridge Univ. Press, 2008.

PROJECT TEAM
Santhosh - 2104251040010 | Adithyan - 2104251040020
FACULTY GUIDE
Mr. Pratham Verma, M.Tech., Assistant Professor

POSTER

END-TO-END WORKFLOW**12 MODEL PERFORMANCE****13 CONFUSION MATRIX****14 KEY FINDINGS**

- Balanced dataset: 20,216 Fake and 20,216 Genuine reviews.
- TF-IDF unigram + bigram features capture useful review-language patterns.
- Multinomial Naive Bayes achieved **89.37%** accuracy on the unseen test set.
- Fake-review precision and recall were both = **89%**.
- Real-time review classification through Streamlit.
- Linguistic indicators are supporting information, not definitive proof of authenticity.

CONCLUSION

The project demonstrates an effective NLP-based approach for identifying potentially fake product reviews. By combining TF-IDF unigram + bigram features with Multinomial Naive Bayes, the system achieved **89.37% test accuracy** on the evaluated dataset. The Streamlit application translates the trained model into an accessible review analysis tool that provides Likely Fake/Likely Genuine predictions and supporting linguistic insights.

16 FUTURE SCOPE

- Analyze complete product pages / product-level review collections.
- Larger, more diverse real-world review datasets.
- Compare additional ML and deep-learning approaches.
- Explore transformer models such as BERT / RoBERTa.
- Large-scale e-commerce recommendation and integration.
- Improve explainability and multilingual analysis.

18 LIMITATIONS

- Classifier analyzes review text; it does not independently verify the reviewer.
- A Likely Fake prediction is not proof that a review is fraudulent.
- Performance depends on the training dataset and its language patterns.
- Current insights focuses primarily on English review text.
- Product-level URL analysis is future scope, not a current core capability.